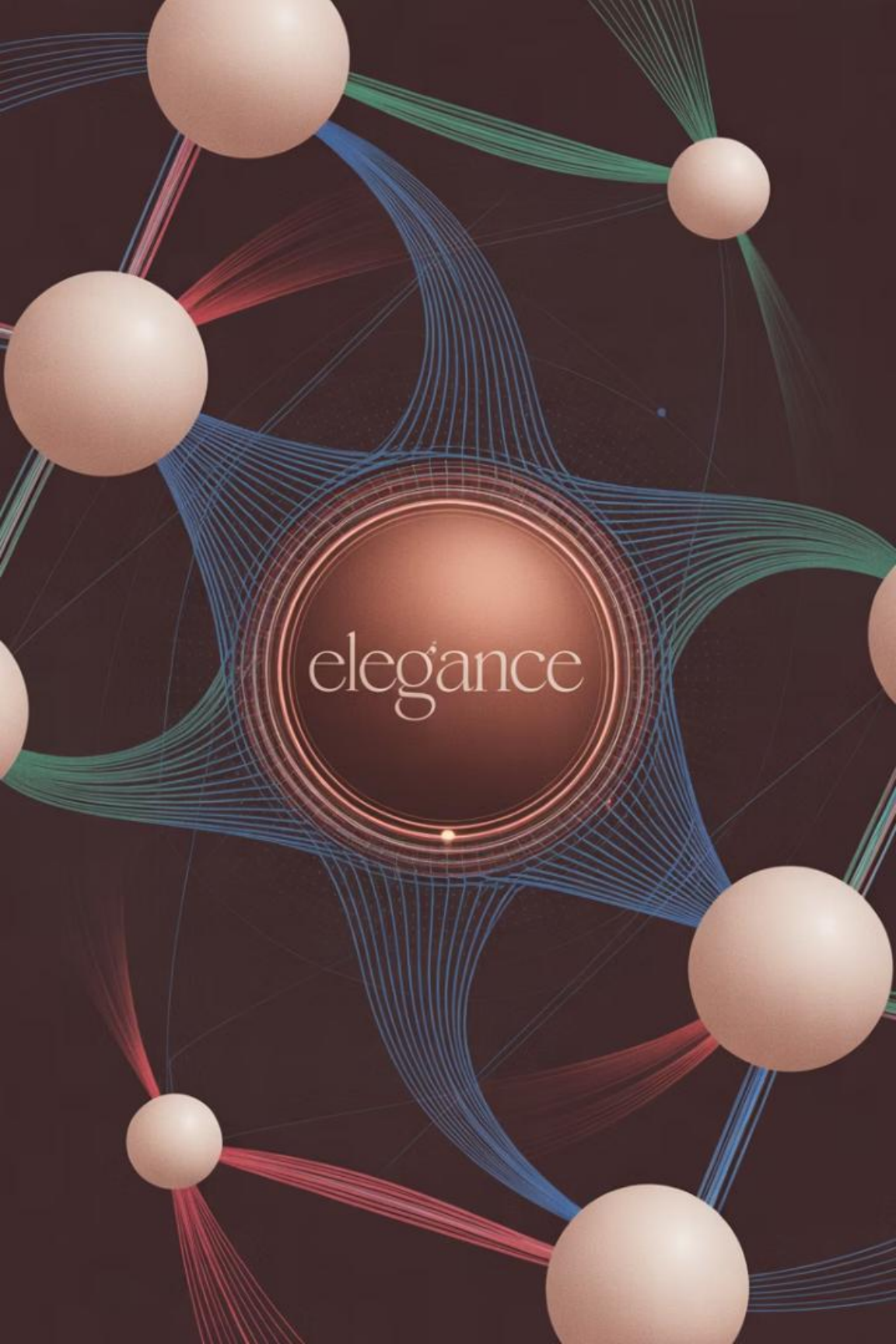




elegance

Conceitos, implementações e aplicações de grafos

Bem-vindo à nossa apresentação sobre grafos! Exploraremos desde conceitos fundamentais até aplicações avançadas desta estrutura poderosa.

Os grafos são a espinha dorsal de inúmeras tecnologias modernas. Eles modelam desde redes sociais até sistemas de transporte complexos.



por Mayana Duarte

O que é um grafo?

Definição Formal

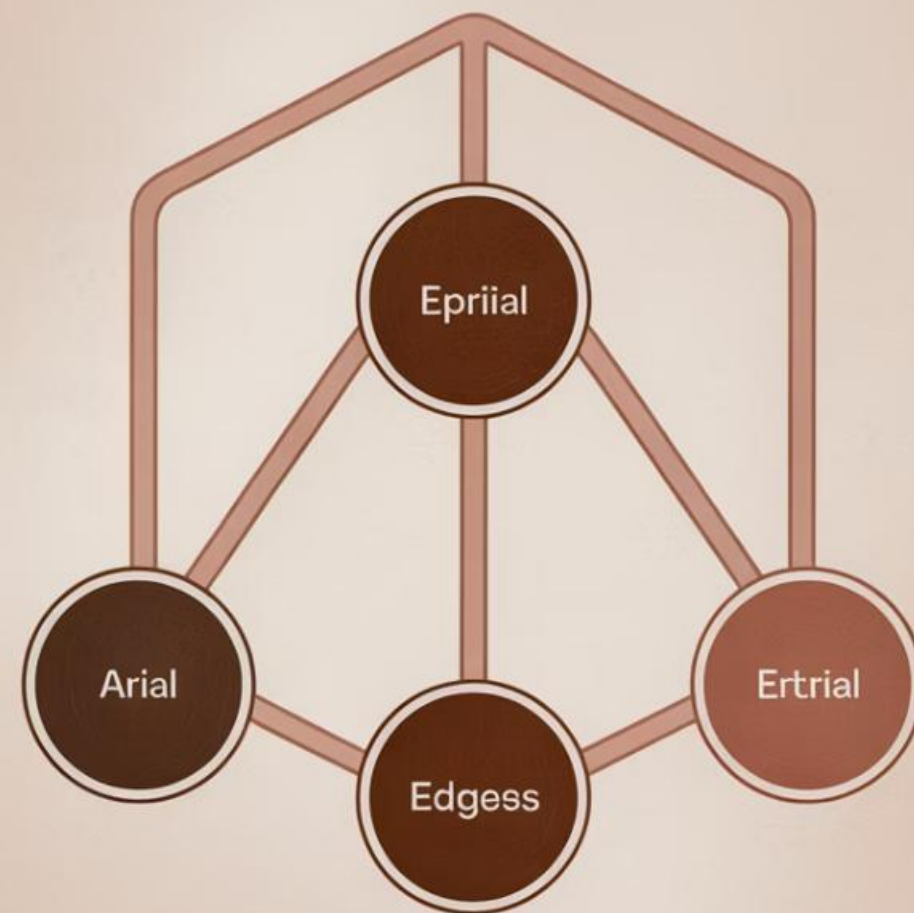
Um grafo G é definido como um par ordenado $G(V,E)$, onde V é um conjunto de vértices e E é um conjunto de arestas.

Vértices (nós)

São os pontos fundamentais do grafo. Podem representar entidades como pessoas, cidades ou dispositivos.

Arestas

São as conexões entre vértices. Representam relações ou ligações como amizades, estradas ou links.



História da teoria dos grafos



1736

Problema das Pontes de Königsberg resolvido por Euler, marcando o nascimento da teoria dos grafos.



Século XIX

Desenvolvimento de teoremas fundamentais sobre coloração de grafos e planaridade.



Século XX

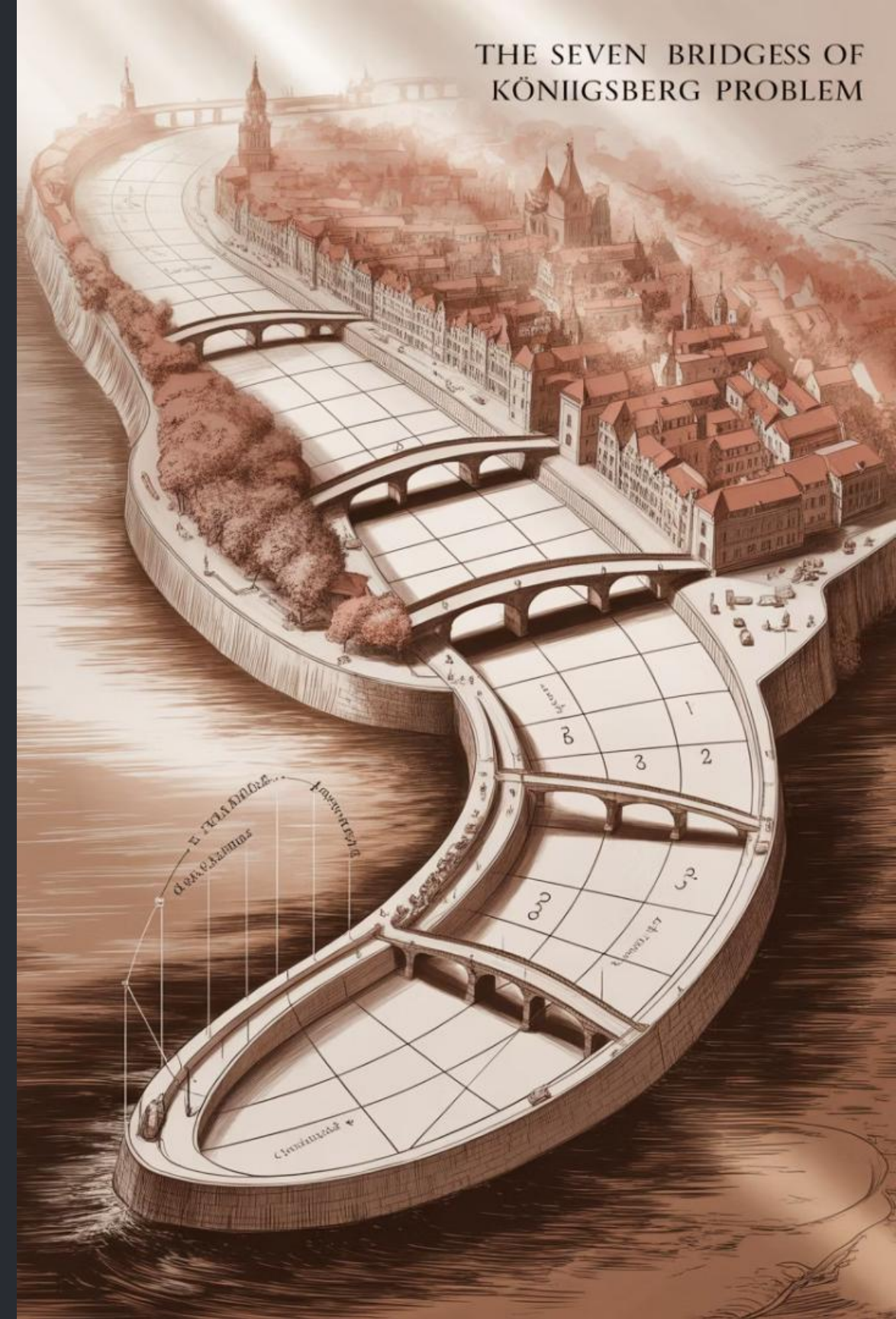
Explosão de aplicações em ciência da computação, transporte e redes de comunicação.



Atualidade

Aplicações em análise de big data, redes sociais e inteligência artificial.

THE SEVEN BRIDGES OF
KÖNIGSBERG PROBLEM



Principais usos dos grafos

Redes Sociais

Modelagem de conexões entre pessoas, análise de influência e detecção de comunidades.

Ciências

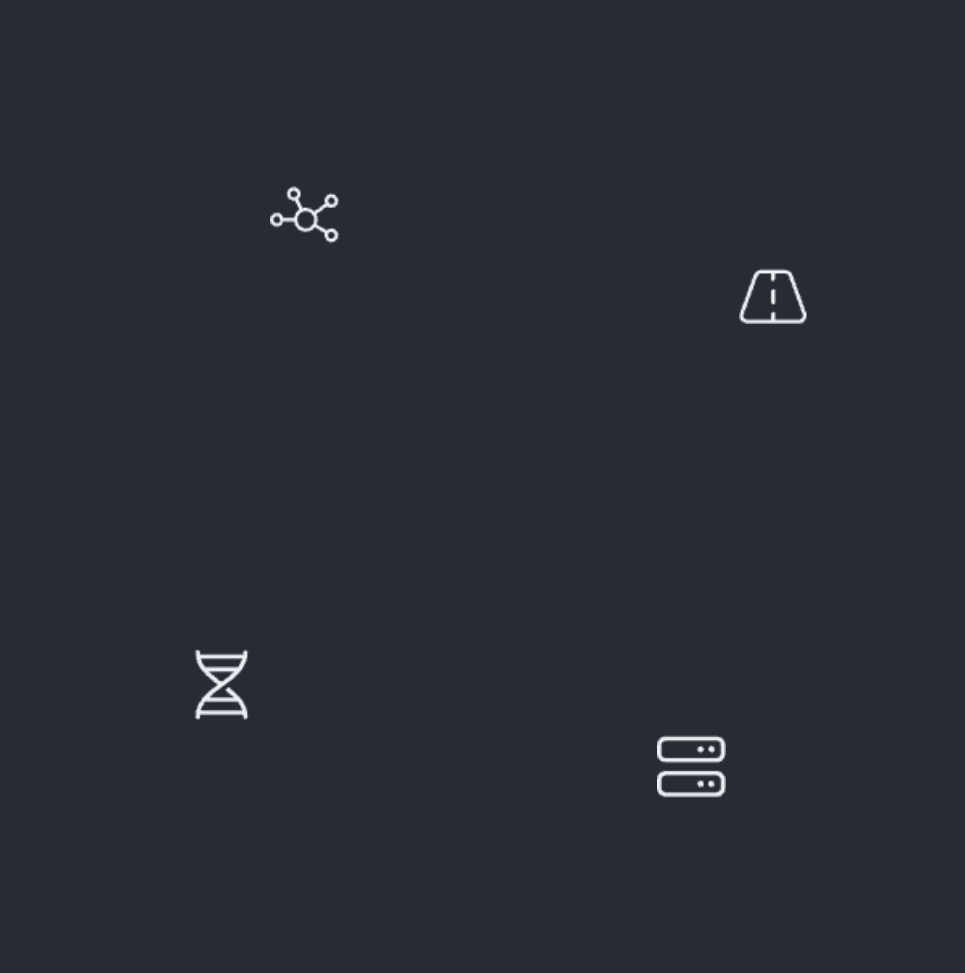
Modelagem de interações moleculares, redes neurais e ecossistemas.

Transportes

Otimização de rotas, planejamento de redes de transporte público e sistemas de navegação.

Organização de Dados

Bancos de dados em grafos, sistemas de recomendação e análise de dependências.



Estruturas básicas: Vértices e arestas

Vértices(Nós)

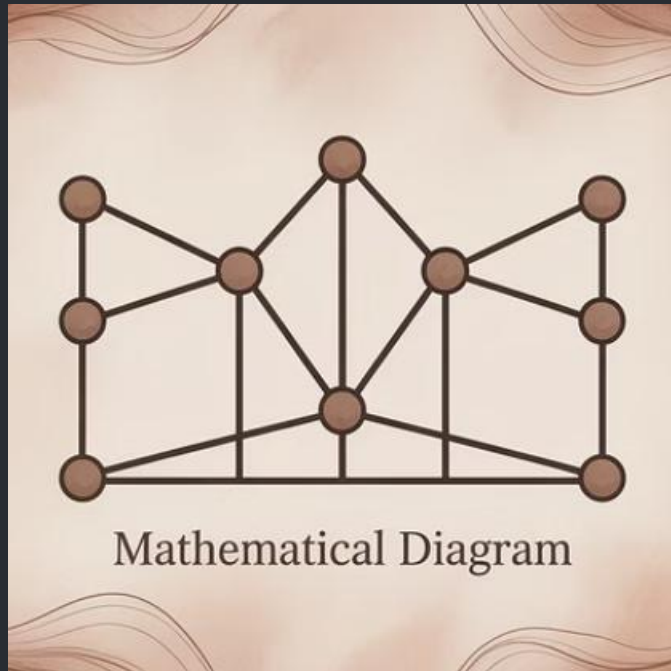
- Entidades fundamentais do grafo
- Podem conter dados ou atributos
- Identificados por rótulos ou índices
- Exemplos: pessoas, cidades, servidores

Arestas(Conexões)

- Ligações entre vértices
- Podem ser direcionadas ou não
- Podem conter pesos ou propriedades
- Exemplos: amizades, estradas, cabos

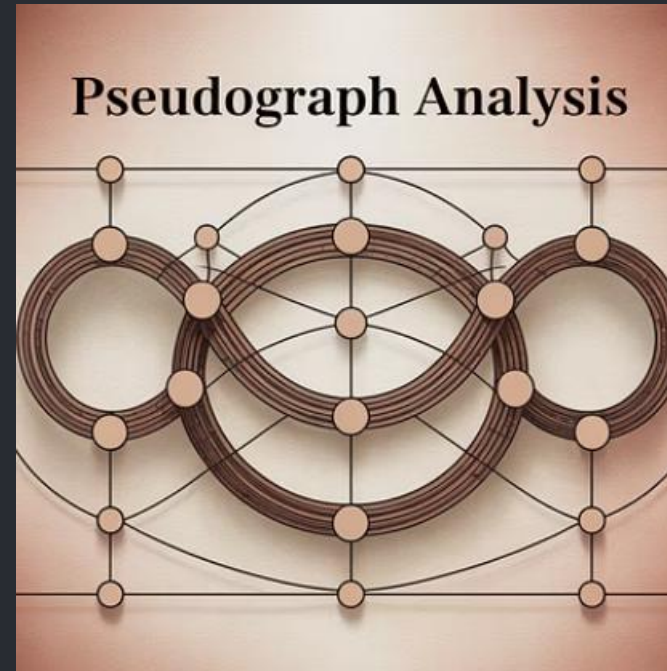
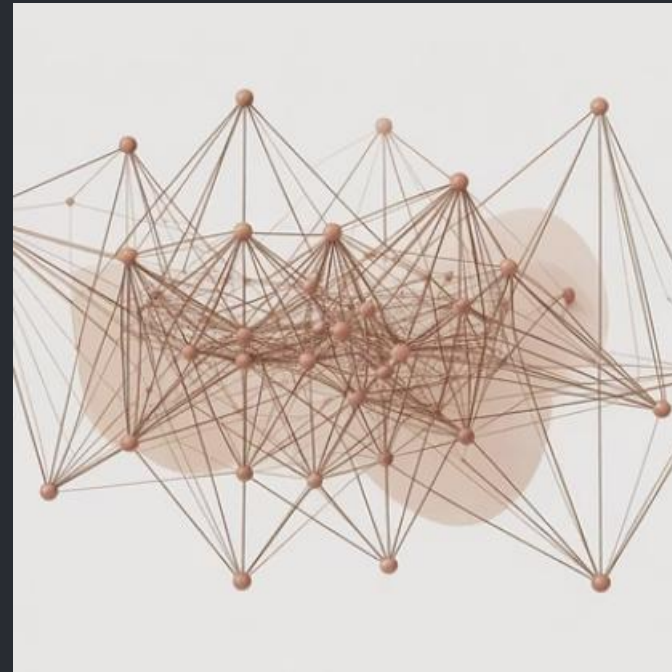


Tipos de grafos



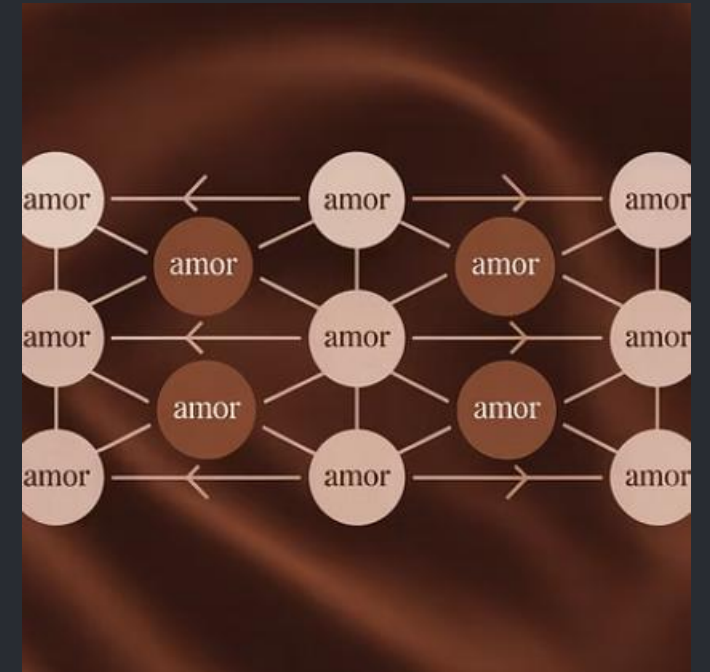
Grafos Simples

Sem arestas paralelas ou loops. Cada aresta conecta dois vértices distintos.



Multigrafos

Permitem múltiplas arestas entre o mesmo par de vértices.



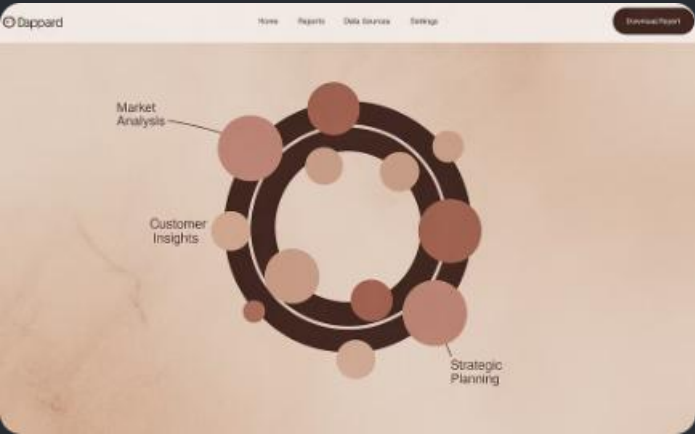
Pseudografos

Permitem loops (arestas que conectam um vértice a ele mesmo).

Grafos Dirigidos

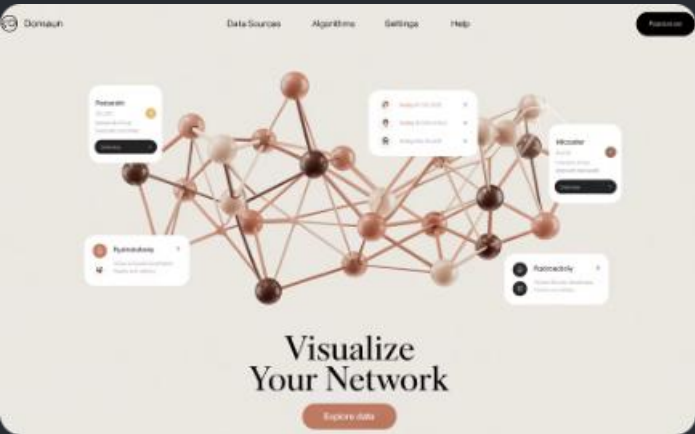
As arestas têm direção, indo de um vértice de origem para um de destino.

Representação visual dos grafos



Layout Circular

Organiza vértices em círculo. Útil para visualizar grafos cíclicos ou regulares.



Layout Baseado em Força

Simula forças físicas entre nós. Destaca agrupamentos naturais no grafo.



Layout Hierárquico

Organiza vértices em níveis. Ideal para árvores e grafos direcionados acíclicos.

Ordem e tamanho do grafo

Ordem do Grafo

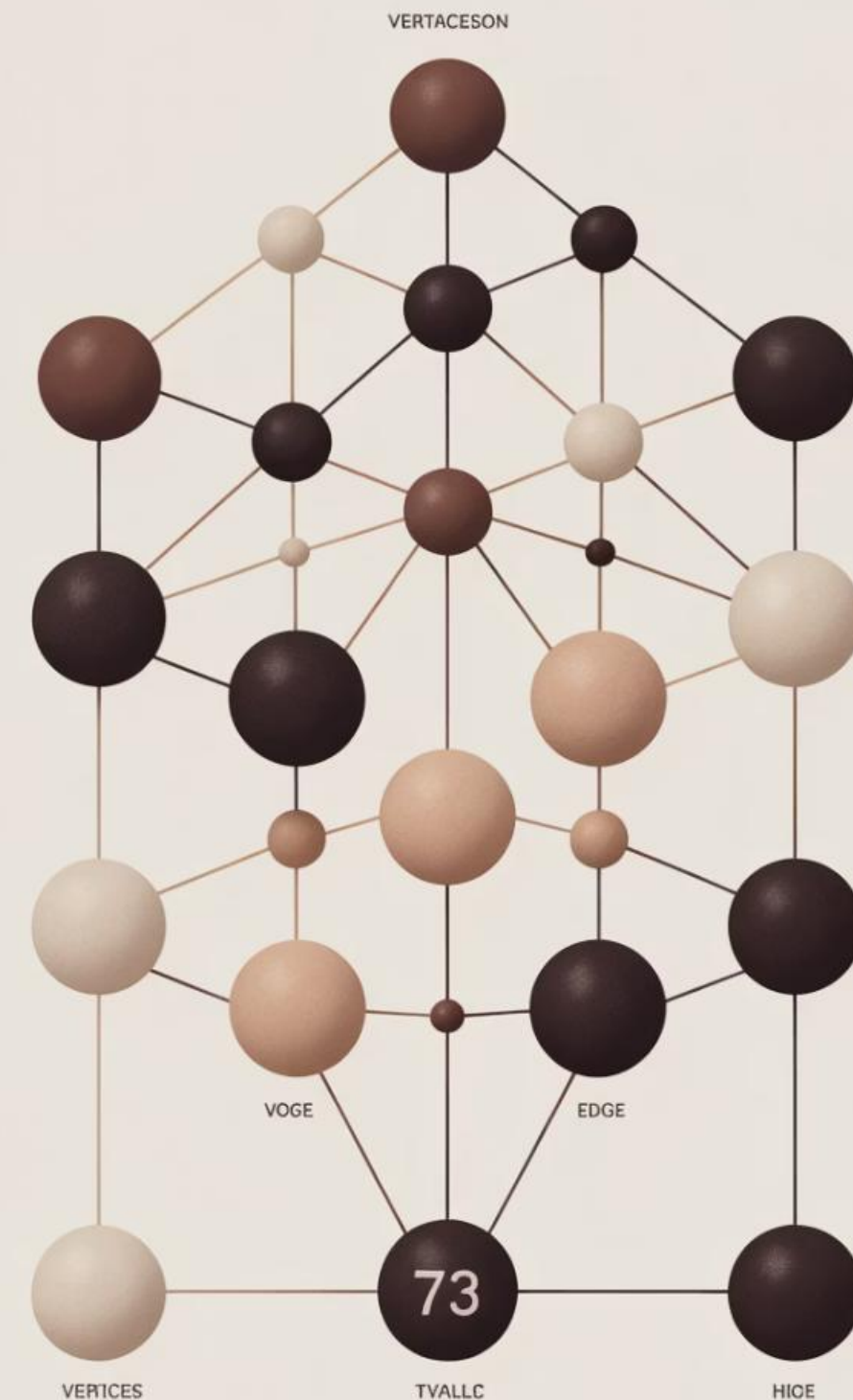
Número total de vértices ($|V|$).
Representa a quantidade de entidades no sistema modelado.

Tamanho do Grafo

Número total de arestas ($|E|$).
Representa a quantidade de relações entre as entidades.

Relações Matemáticas

Em um grafo simples, o máximo de arestas possíveis é $n(n-1)/2$, onde n é o número de vértices.



Conceitos essenciais



Grau dos Vértices

Número de arestas conectadas a um vértice. Em grafos dirigidos, distinguimos grau de entrada e saída.



Vizinhança

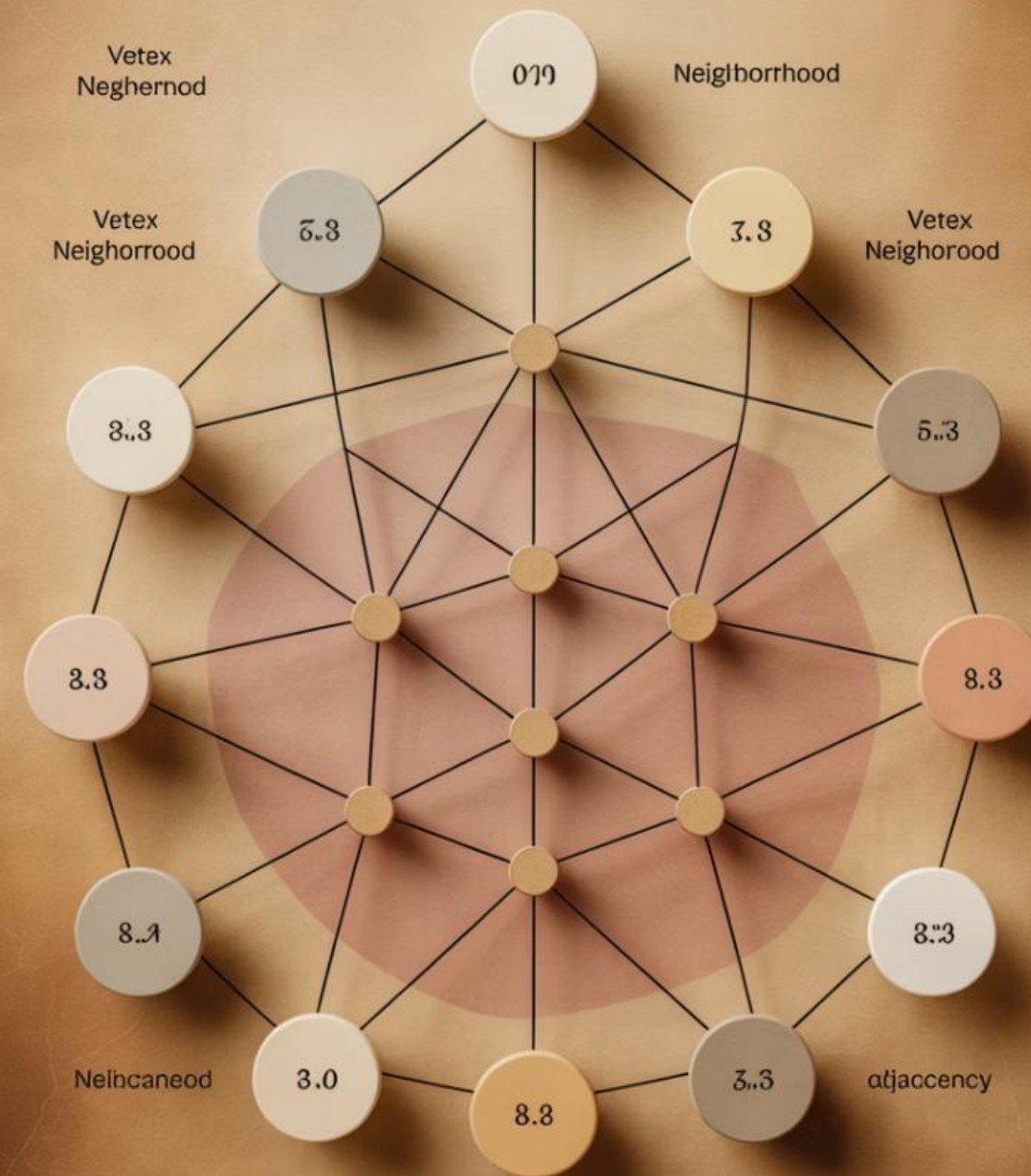
Conjunto de vértices adjacentes a um determinado vértice. A vizinhança de v é denotada por $N(v)$.



Adjacência

Dois vértices são adjacentes se existe uma aresta conectando-os diretamente.

Vertex Degree, Neighborhood, and Adjacency



Tipos especiais de grafos



Árvores

Grafos conexos sem ciclos



Ciclos

Grafos onde cada vértice tem grau 2



Grafos Completos

Todos os vértices são adjacentes entre si



Grafos Bipartidos

Vértices divididos em dois conjuntos



Grafos Regulares

Todos os vértices têm o mesmo grau

Subgrafos e componentes

Subgrafos

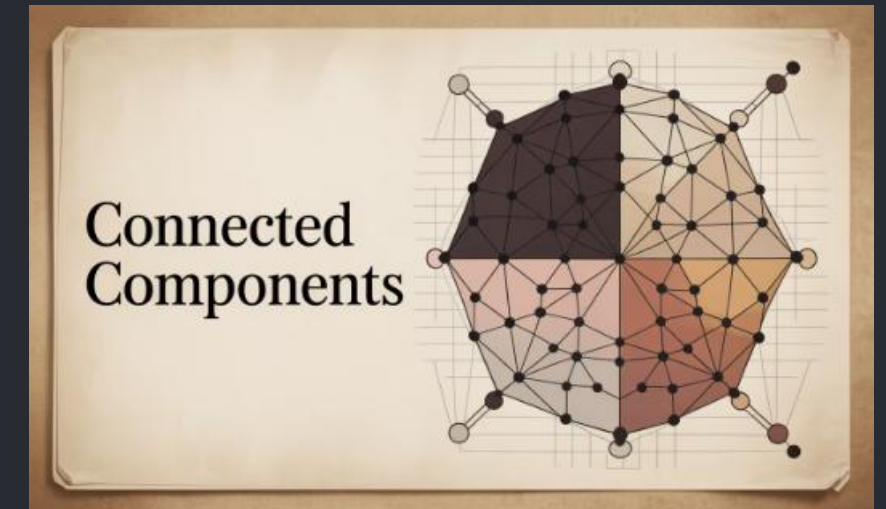
Um subgrafo H de um grafo G é formado por um subconjunto de vértices e arestas de G .

Subgrafos mantêm todas as propriedades do grafo original para os elementos selecionados.

Componentes Conexas

São subgrafos máximos onde existe um caminho entre quaisquer dois vértices.

Um grafo conexo possui apenas uma componente. Grafos desconexos têm múltiplas componentes.



Caminhos e ciclos



Caminho

Sequência de vértices conectados por arestas, sem repetição de arestas



Trilha

Caminho onde vértices podem se repetir, mas arestas não



Ciclo

Caminho fechado onde o primeiro e último vértices são idênticos

Grafos conexos e fortemente conexos

Grafos Conexos

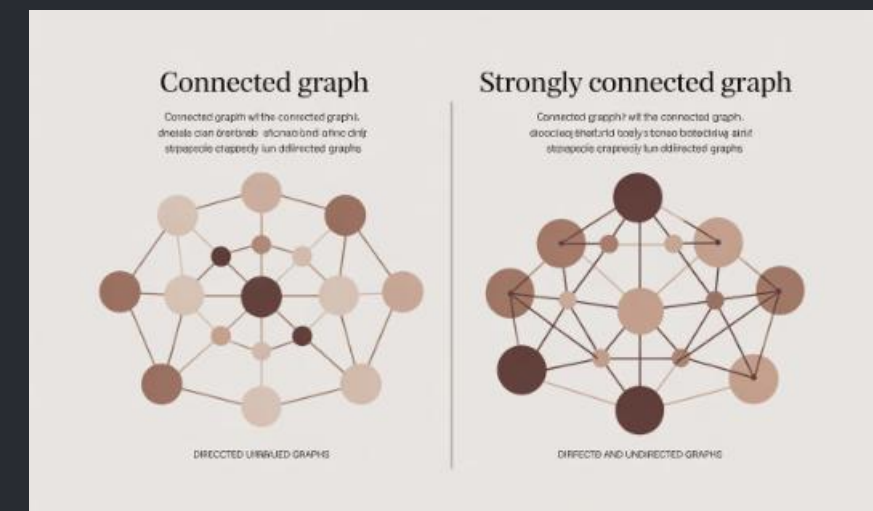
Um grafo não-dirigido é conexo se existe um caminho entre quaisquer dois vértices.

A conectividade é uma propriedade fundamental em muitas aplicações práticas.

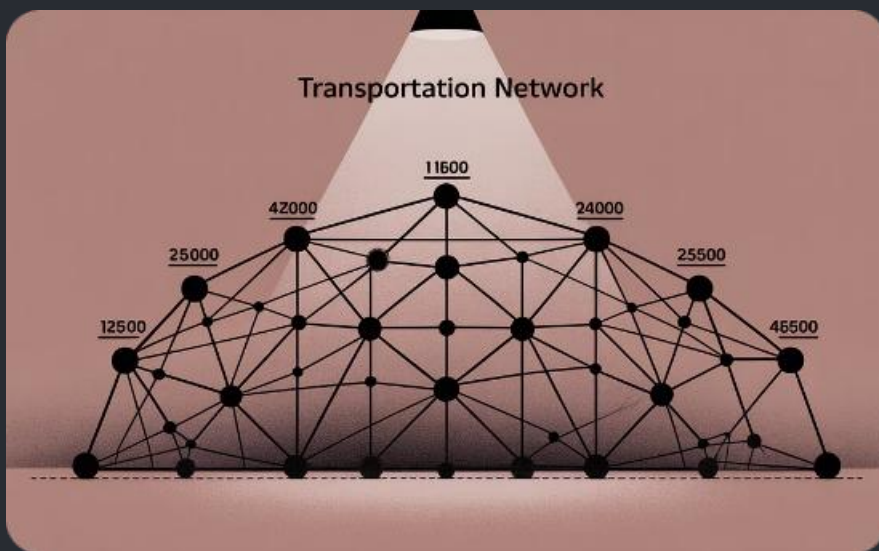
Grafos Fortemente Conexos

Um grafo dirigido é fortemente conexo se existe um caminho dirigido entre quaisquer par de vértices.

Aplicações: análise de acessibilidade em redes de comunicação.

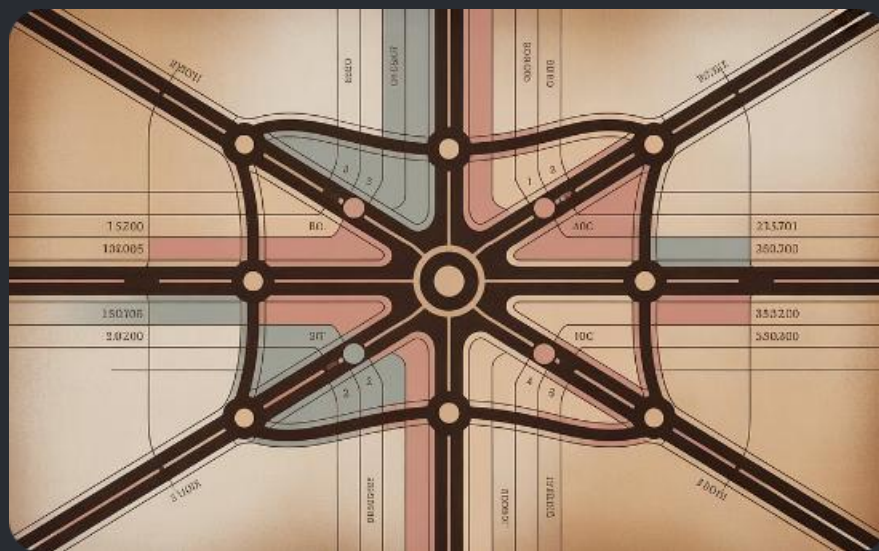


Grafos ponderados



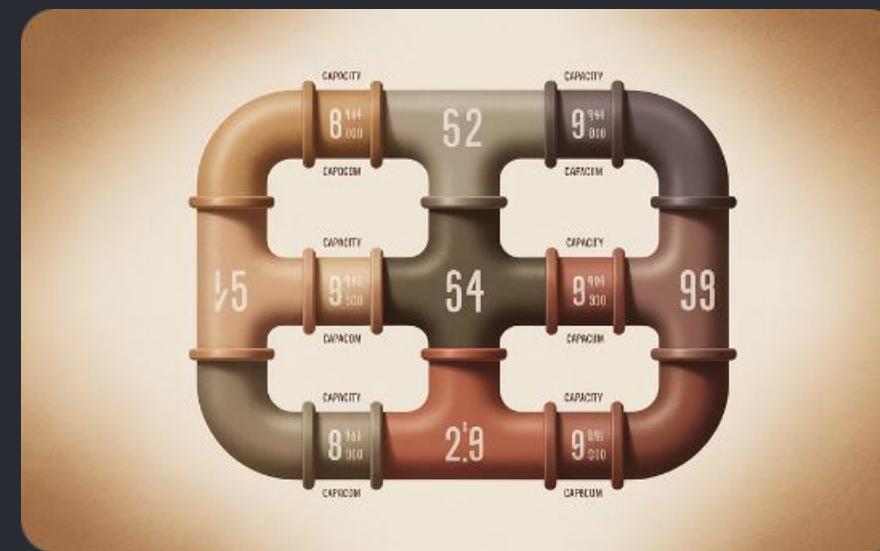
Arestas com Pesos

Em grafos ponderados, cada aresta possui um valor numérico associado. Este valor representa custos, distâncias ou capacidades.



Redes de Transporte

Os pesos podem representar distâncias entre cidades, tempos de viagem ou custos de deslocamento.



Fluxo em Redes

Os pesos podem indicar capacidades máximas em sistemas de distribuição, como redes elétricas ou hidráulicas.

Grafos orientados vs. não-orientados



Grafos Não-Orientados

As arestas não possuem direção. A relação entre vértices é simétrica e bidirecional.



Grafos Orientados (Dígrafos)

As arestas possuem direção. A relação entre vértices tem origem e destino definidos.



Aplicações Típicas

Não-orientados: redes sociais de amizade. Orientados: fluxo de tráfego, hierarquias.

Grafos simples vs. multigrafos

Grafos Simples

Não possuem loops (arestas que conectam um vértice a ele mesmo).

Não possuem arestas paralelas (múltiplas arestas entre o mesmo par de vértices).

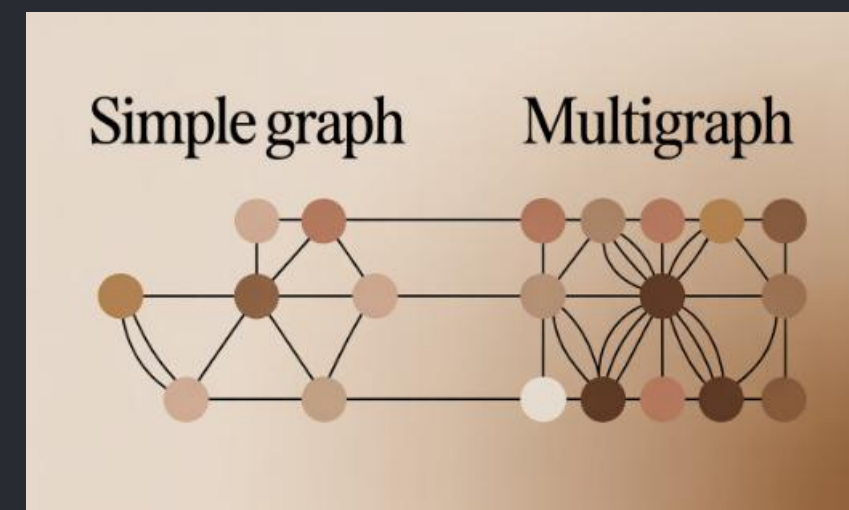
Aplicações: redes sociais, onde uma relação entre pessoas é única.

Multigrafos

Permitem arestas paralelas entre o mesmo par de vértices.

Úteis para representar múltiplas conexões de diferentes tipos.

Aplicações: rotas de transporte entre cidades, com diferentes meios.



Implementação computacional: Visão geral

Estruturas de Dados para Grafos

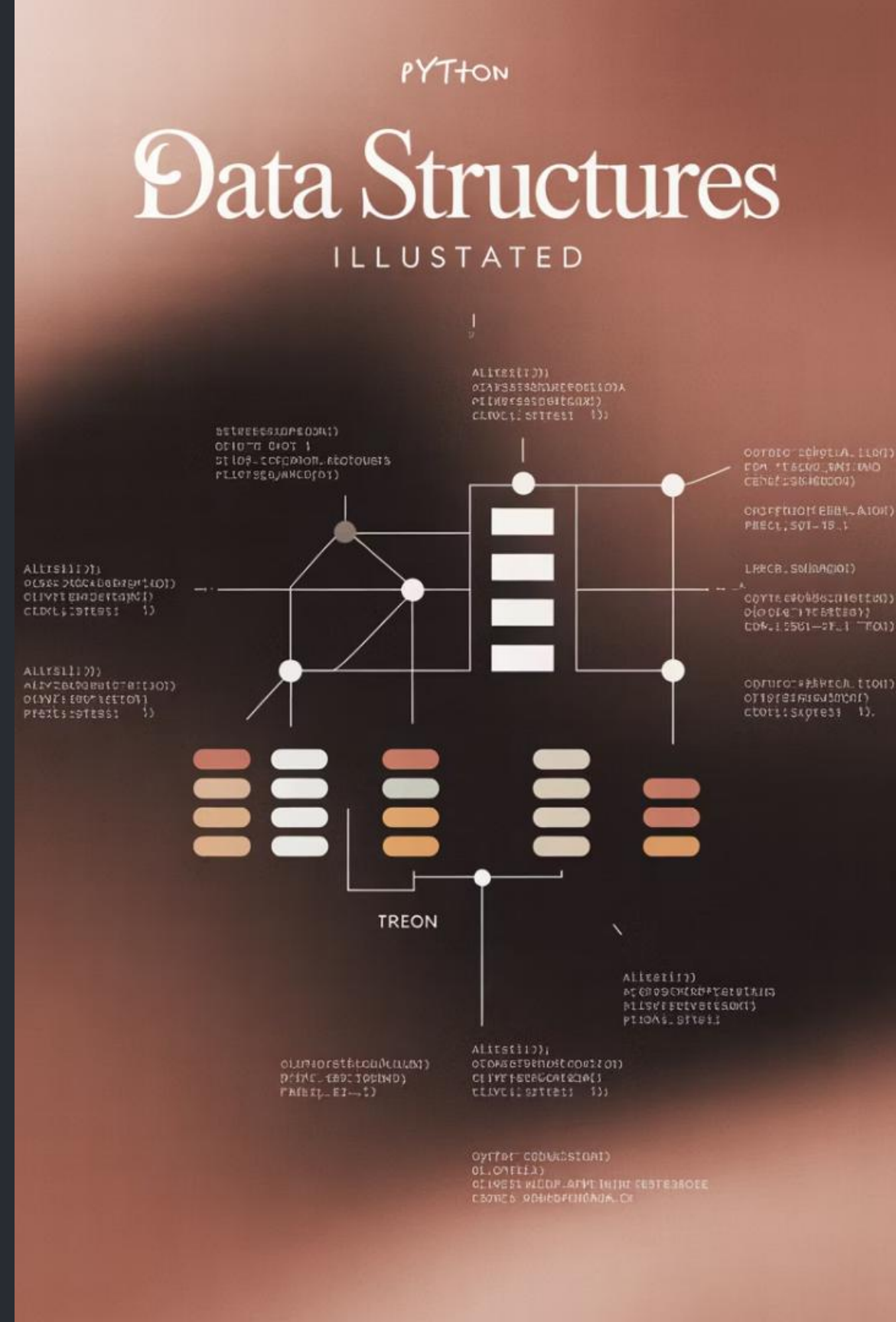
A escolha da estrutura depende das operações mais frequentes e das características do grafo.

Considerações de Complexidade

Balanceamento entre tempo de acesso e consumo de memória na representação escolhida.

Tipos de Implementação

Lista de adjacência, matriz de adjacência, matriz de incidência e representações híbridas.



Representação por lista de adjacência



Conceito

Cada vértice mantém uma lista de seus vizinhos adjacentes



Vantagens

Eficiente em espaço para grafos esparsos

3

Desvantagens

Verificação de adjacência mais lenta

Exemplo de lista de adjacência

Grafo de Exemplo

Um grafo com 5 vértices (A, B, C, D, E) e 7 arestas.

As conexões são: A-B, A-C, B-C, B-D, C-D, C-E, D-E.

Representação em Lista

A → [B, C]

B → [A, C, D]

C → [A, B, D, E]

D → [B, C, E]

E → [C, D]



Representação por matriz de adjacência

Estrutura da Matriz



Matriz quadrada $n \times n$, onde n é o número de vértices. O valor na posição $[i,j]$ indica se existe aresta entre os vértices i e j .

Vantagens

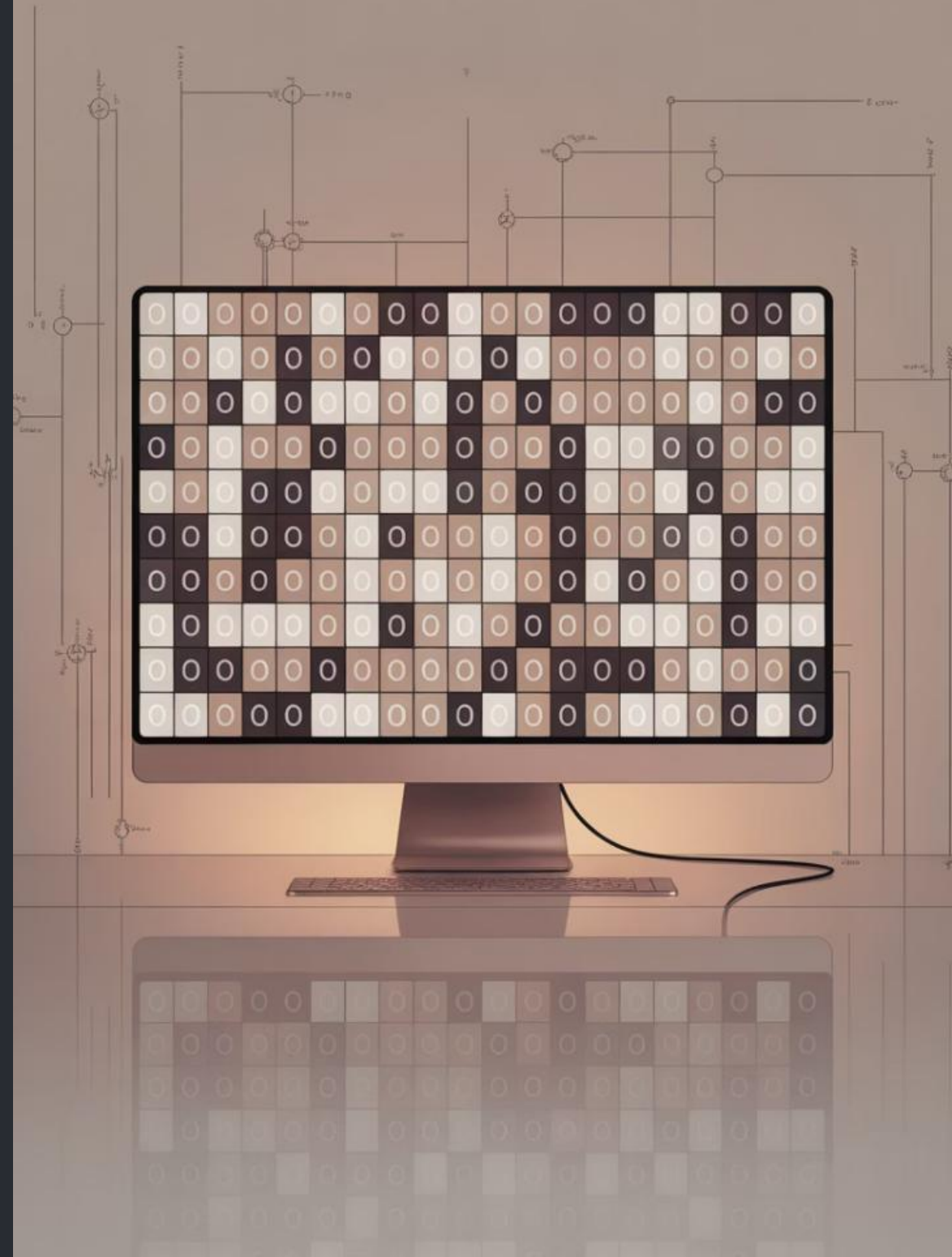
2

Verificação rápida de adjacência ($O(1)$). Simples de implementar e entender.

Desvantagens

3

Alta utilização de memória para grafos esparsos ($O(V^2)$).



Exemplo de matriz de adjacência

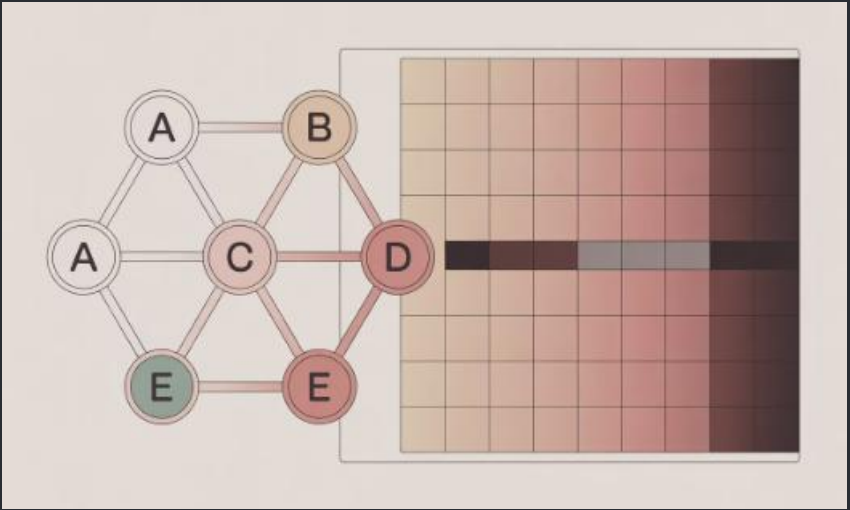
Grafo de Exemplo

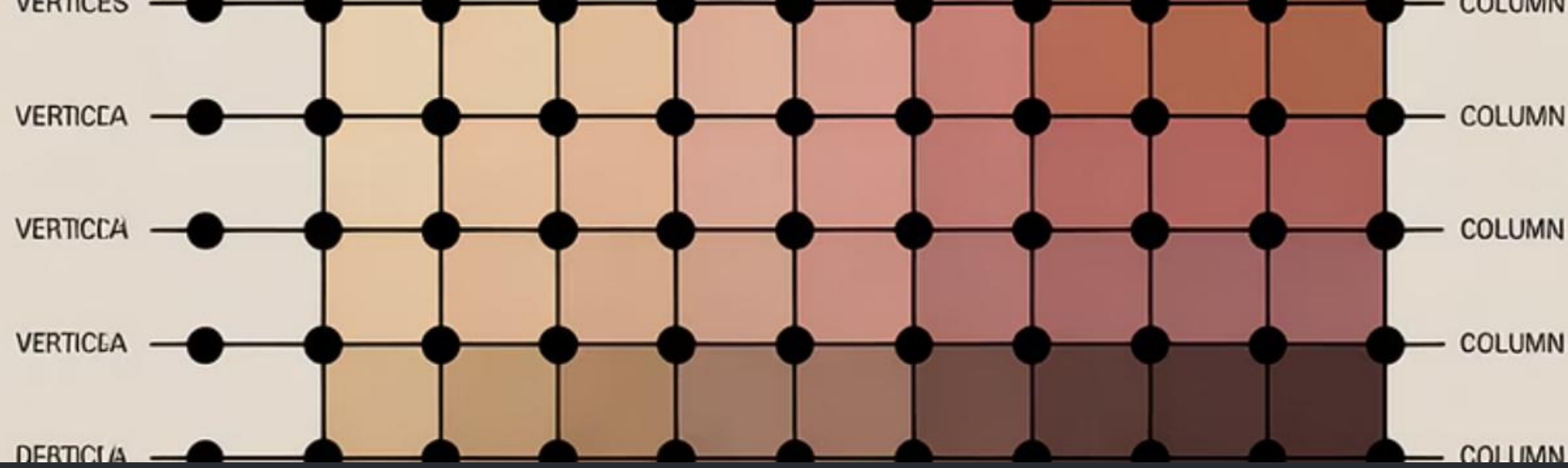
Um grafo com 5 vértices (A, B, C, D, E) e 7 arestas.

As conexões são: A-B, A-C, B-C, B-D, C-D, C-E, D-E.

Matriz de Adjacência

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	1	1	0
C	1	1	0	1	1
D	0	1	1	0	1
E	0	0	1	1	0





Outras representações: matriz de incidência



Conceito

Matriz onde linhas representam vértices e colunas representam arestas. Valores indicam incidência.



Utilidade

Útil para problemas de fluxo em rede e programação linear.



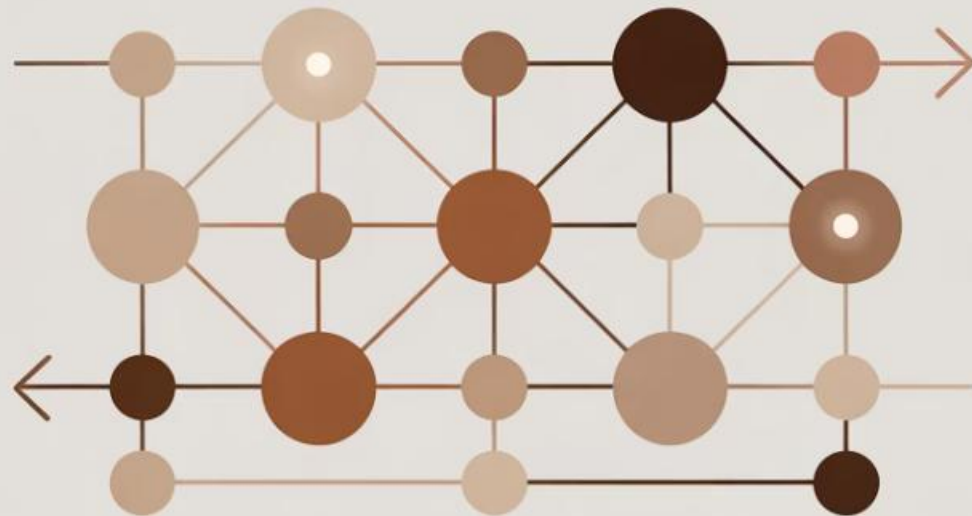
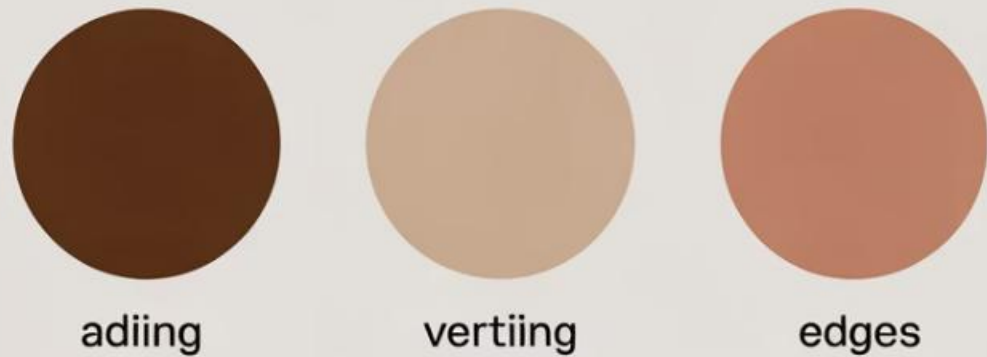
Estrutura

Em grafos não-dirigidos: 1 se incidente, 0 caso contrário. Em dirigidos: -1 (origem), 1 (destino).

Escolhendo a implementação apropriada



Graph Operations



Principais operações em grafos



Adicionar

Inserção de novos vértices ou arestas no grafo.



Remover

Exclusão de vértices ou arestas existentes.



Consultar

Verificação de adjacência e vizinhança.



Percorrer

Visitar todos os vértices e arestas do grafo.

Algoritmos básicos em grafos: visão geral

Algoritmos de Busca

BFS (Busca em Largura) e DFS (Busca em Profundidade) para percorrer grafos.

Análise de Componentes

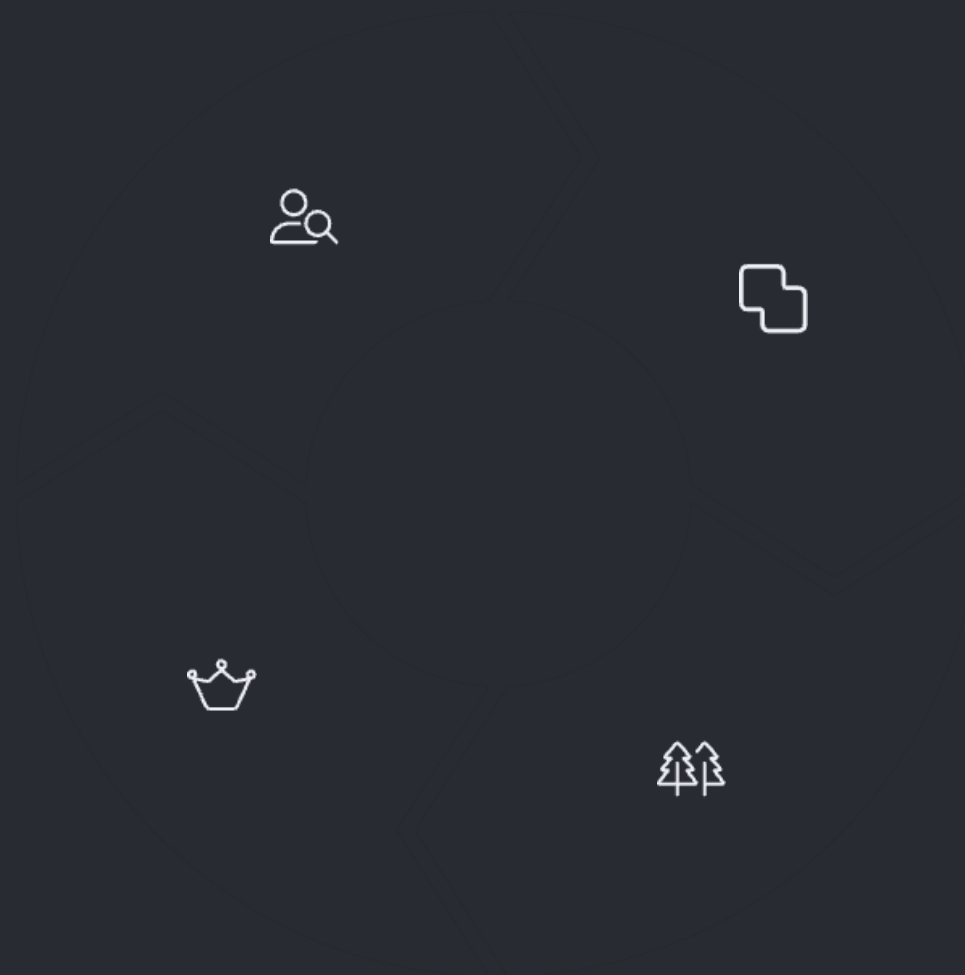
Deteção de componentes conexas e componentes fortemente conexas.

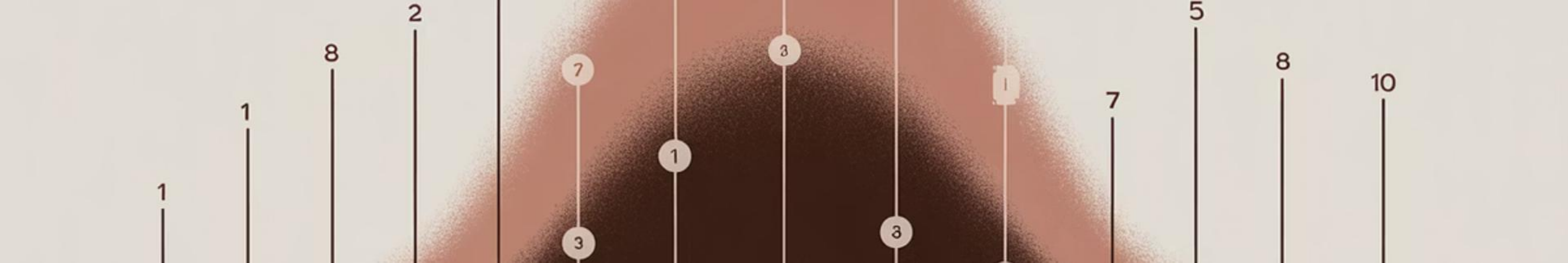
Caminhos Mínimos

Dijkstra, Bellman-Ford e Floyd-Warshall para encontrar rotas ótimas.

Árvores Geradoras

Kruskal e Prim para árvores geradoras mínimas (MST).





Busca em largura (BFS)



Funcionamento

Utiliza uma fila para visitar primeiro os vértices mais próximos da origem. Expande em "ondas" a partir do vértice inicial.



Complexidade

$O(V+E)$ para tempo e espaço, onde V é o número de vértices e E o número de arestas.



Aplicações

Encontrar caminhos mais curtos em grafos não ponderados. Análise de redes sociais. Resolução de quebra-cabeças.

Exemplo prático: BFS

Início

Começa no vértice inicial A. Adiciona A à fila e marca como visitado.

Primeira Iteração

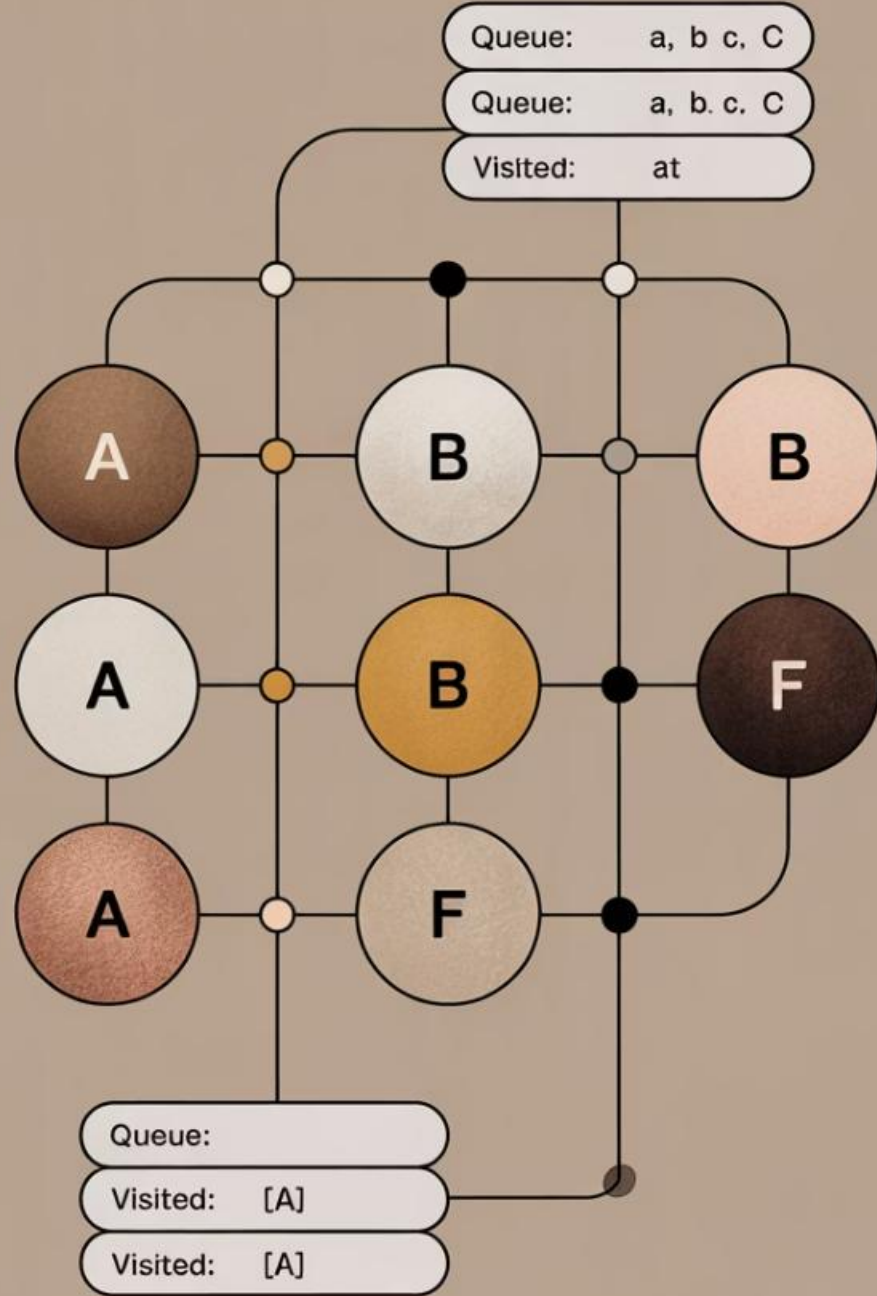
Remove A da fila. Visita todos os vizinhos não visitados de A (B e C). Adiciona B e C à fila.

Segunda Iteração

Remove B da fila. Visita todos os vizinhos não visitados de B (D). Adiciona D à fila.

Iterações Finais

Continua o processo até que a fila esteja vazia. Ordem final de visita: A, B, C, D, E, F.



Busca em profundidade (DFS)



Funcionamento

Utiliza uma pilha (ou recursão) para explorar um caminho até o fim antes de retroceder.

Complexidade

$O(V+E)$ para tempo e $O(V)$ para espaço na versão recursiva.

Utilidade

Detecção de ciclos, ordenação topológica, encontrar componentes conexas.

Implementação

Geralmente implementado de forma recursiva, mas pode usar uma pilha explícita.



Exemplo prático: DFS

Início

Começa no vértice inicial A. Marca A como visitado e o coloca na pilha.

Exploração em Profundidade

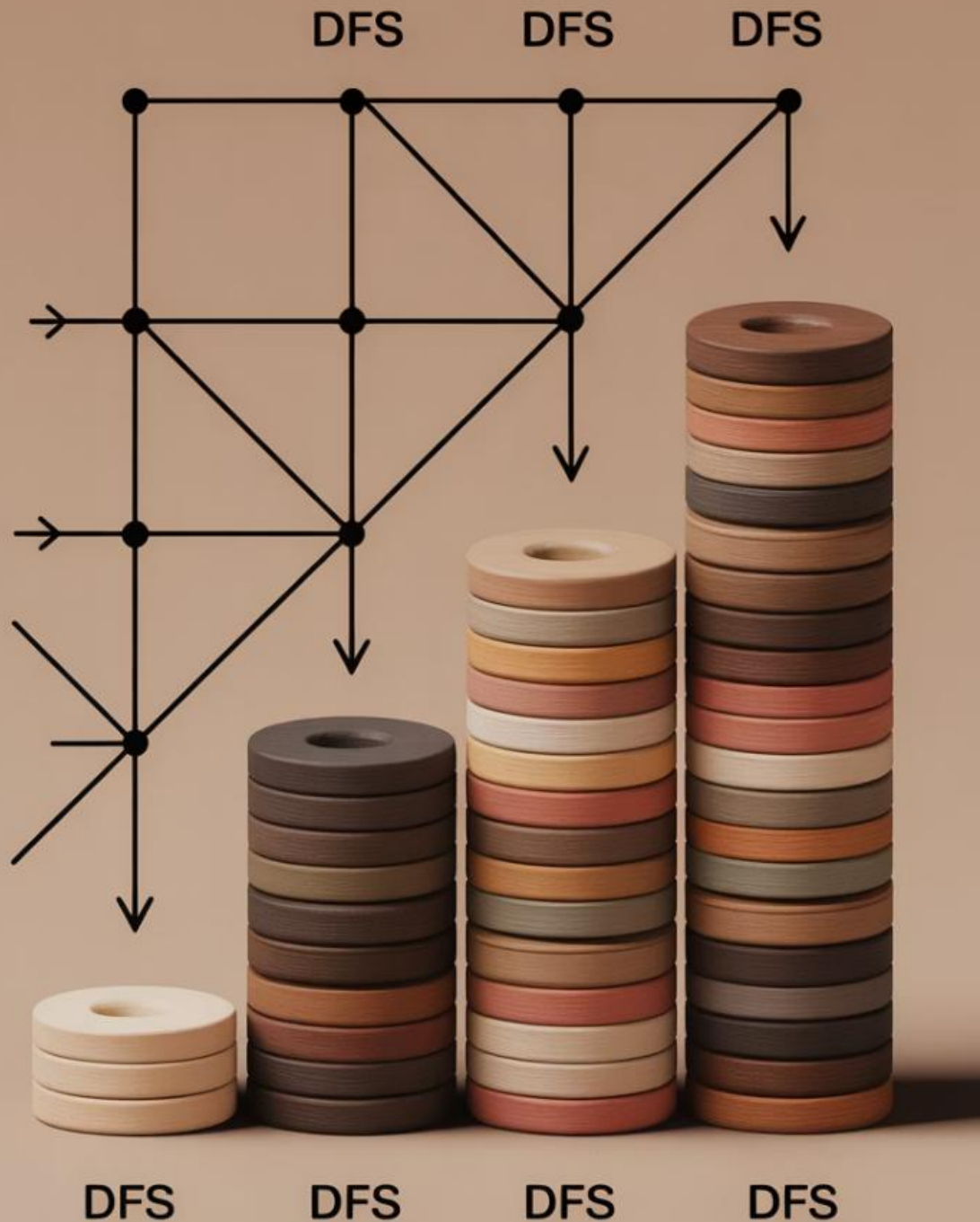
Visita o primeiro vizinho não visitado de A (B). Marca B e o coloca na pilha. Continua com o primeiro vizinho de B.

Backtracking

Quando um vértice não tem vizinhos não visitados, retrocede (faz backtracking) para o vértice anterior na pilha.

Conclusão

Continua até que todos os vértices tenham sido visitados. Ordem final: A, B, D, F, E, C.



Comparação: BFS vs. DFS

Busca em Largura (BFS)

- Usa estrutura de fila
- Encontra caminhos mais curtos em grafos não ponderados
- Explora o grafo em "ondas" concêntricas
- Melhor para: procurar nós próximos da origem

Busca em Profundidade (DFS)

- Usa estrutura de pilha ou recursão
- Boa para explorar todos os caminhos possíveis
- Explora o grafo seguindo um caminho até o fim
- Melhor para: detecção de ciclos, ordenação topológica



Algoritmo de Dijkstra

Propósito

Encontra o caminho mais curto entre um vértice de origem e todos os outros vértices em um grafo ponderado com pesos não negativos.

Funcionamento

Mantém uma fila de prioridade com vértices. A cada passo, seleciona o vértice com menor distância acumulada.

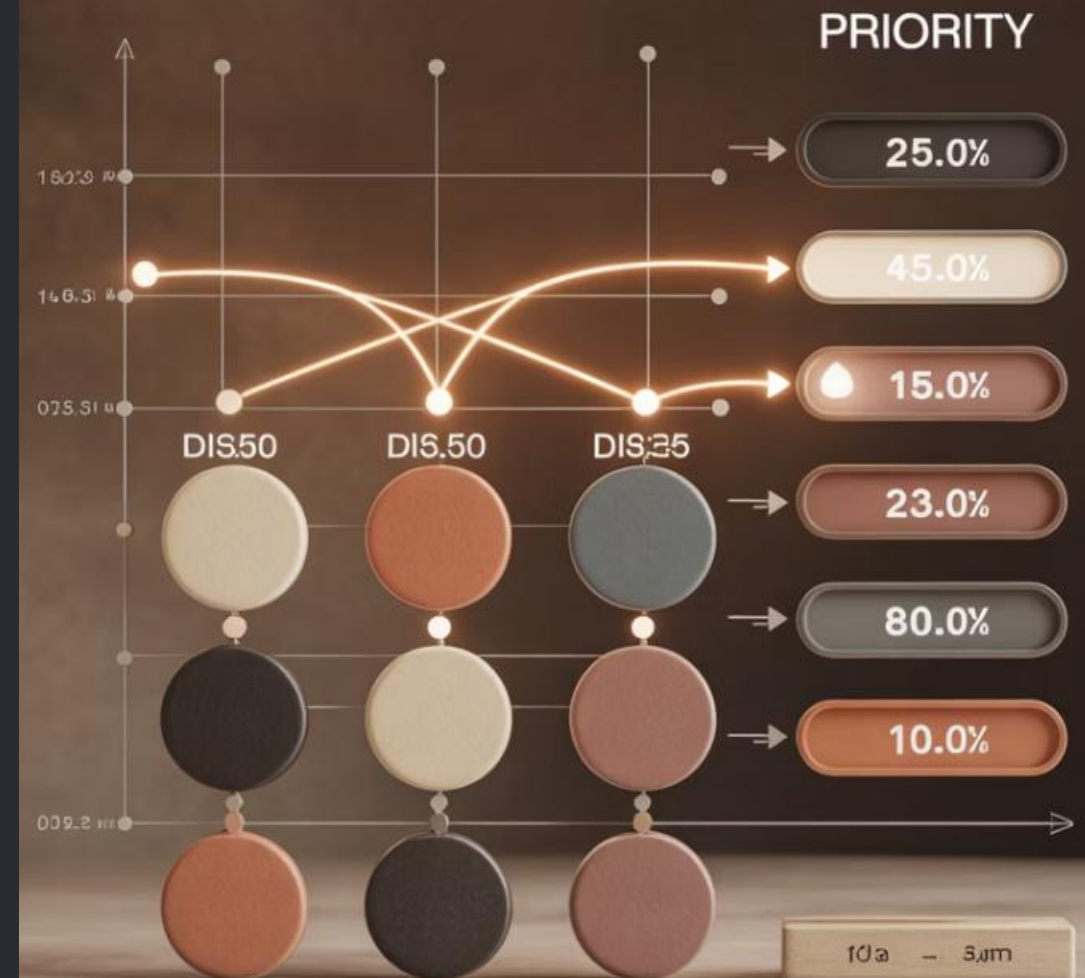
Complexidade

$O(V^2)$ com matriz de adjacência ou $O((V+E)\log V)$ com fila de prioridade.

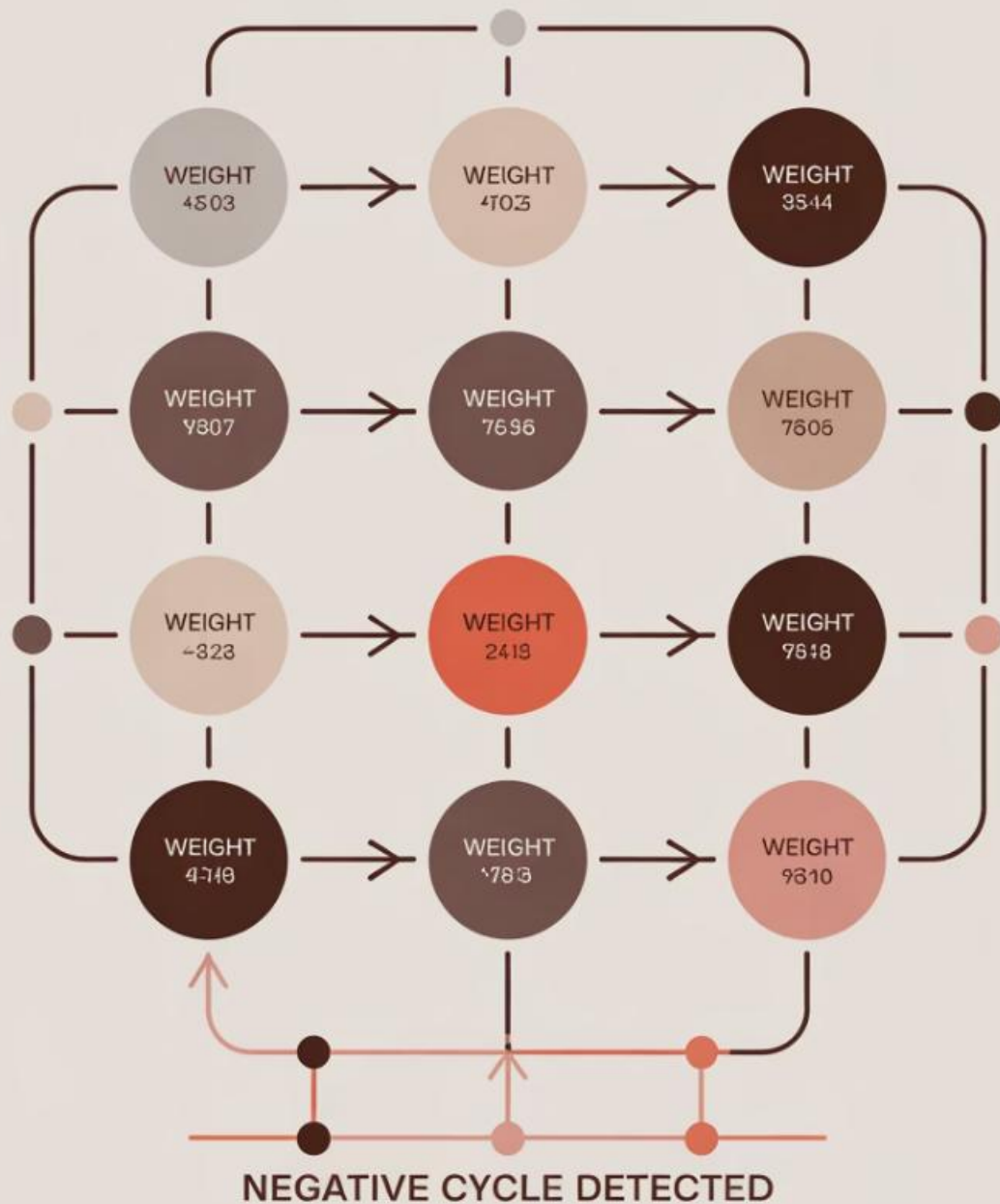
Limitação

Não funciona corretamente em grafos com pesos negativos nas arestas.

Dijkstra Algorithm



Bellman-Ford Algorithm



Algoritmo de Bellman-Ford

Propósito

Encontra o caminho mais curto em grafos ponderados, mesmo com pesos negativos.

Vantagem sobre Dijkstra

Suporta pesos negativos e detecta ciclos negativos (situações onde não há caminho mínimo).

Complexidade

$O(V \cdot E)$, menos eficiente que Dijkstra, mas mais versátil quanto aos pesos.

Aplicações

Roteamento de pacotes, análise financeira onde perdas são representadas como pesos negativos.

Busca de caminhos em grafos não ponderados

BFS para Caminhos Mais Curtos

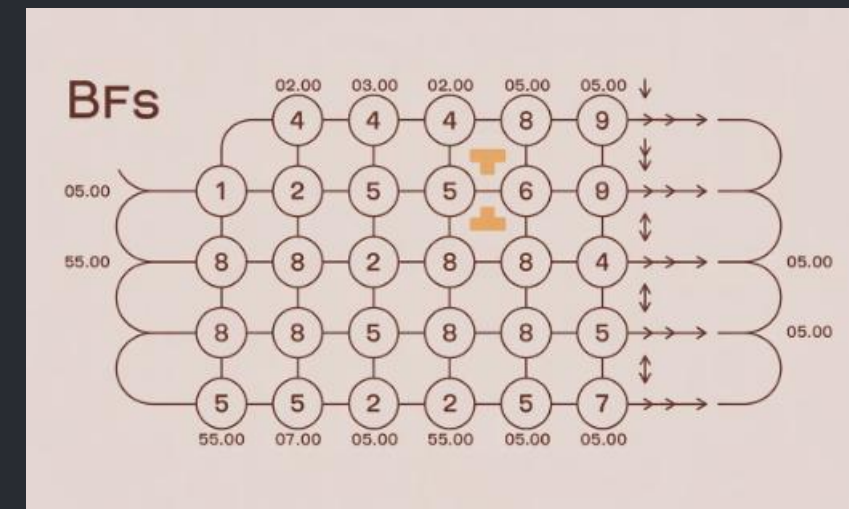
Em grafos não ponderados, a BFS garante encontrar o caminho com menor número de arestas.

Cada "nível" da BFS corresponde à distância (número de arestas) desde a origem.

Implementação

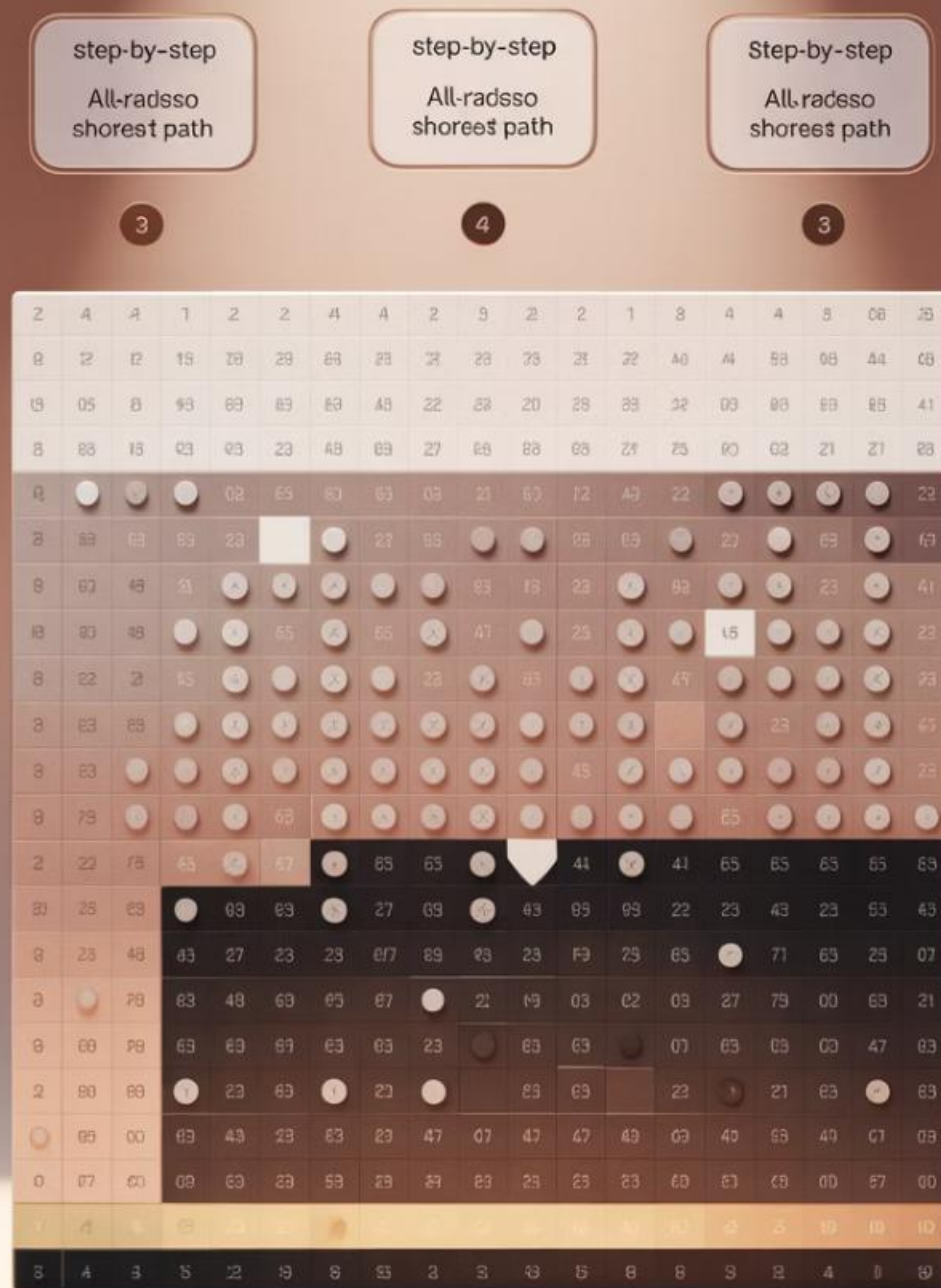
Além da fila tradicional, mantém-se um array de distâncias e outro de predecessores.

Ao final, é possível reconstruir o caminho mínimo da origem até qualquer vértice.



Floyd-Warshall

Shortest Paths



Algoritmo de Floyd-Warshall

Propósito

Encontra os caminhos mais curtos entre todos os pares de vértices de um grafo ponderado.

Abordagem

Usa programação dinâmica para considerar todos os possíveis vértices intermediários em um caminho.

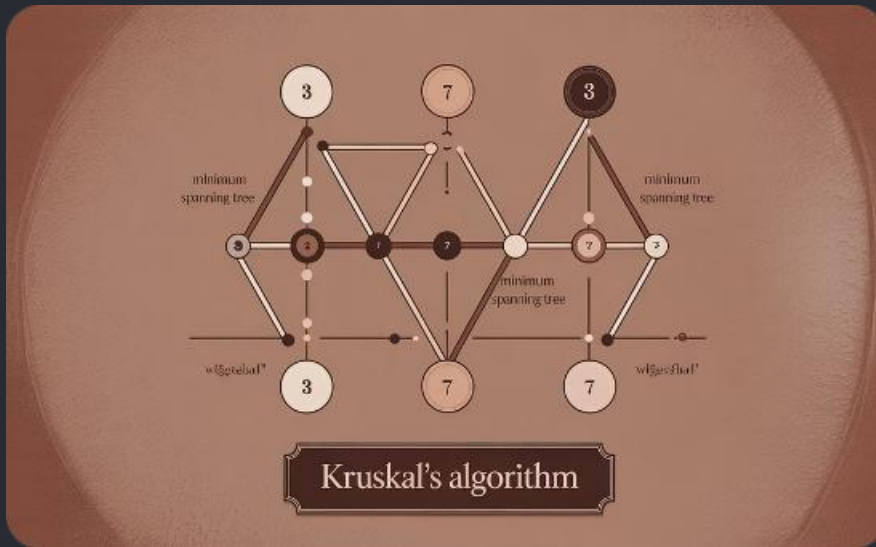
Complexidade

$O(V^3)$, onde V é o número de vértices. Eficiente para grafos pequenos e densos.

Vantagens

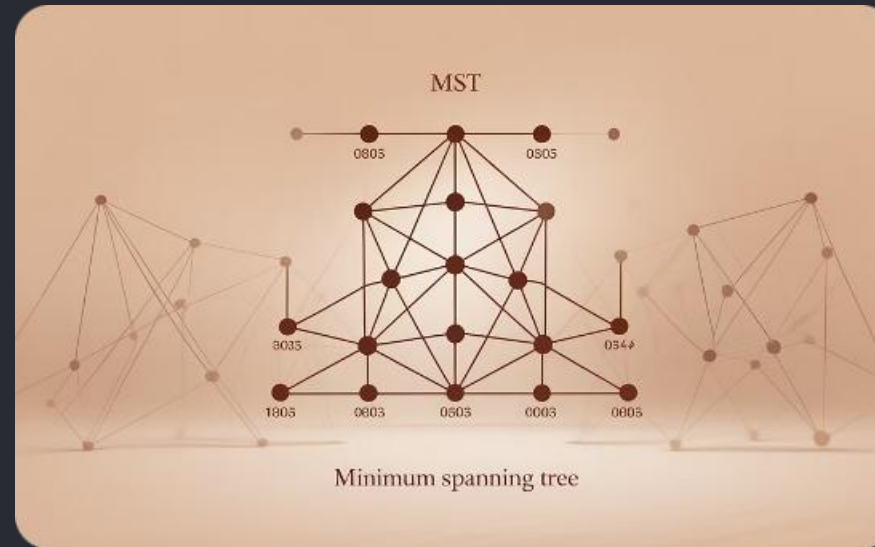
Implementação simples. Funciona com pesos negativos (desde que não haja ciclos negativos).

Algoritmos de árvores geradoras mínimas (MST)



Algoritmo de Kruskal

Constrói a MST adicionando arestas em ordem crescente de peso, evitando ciclos. Usa estrutura Union-Find para detecção eficiente de ciclos.



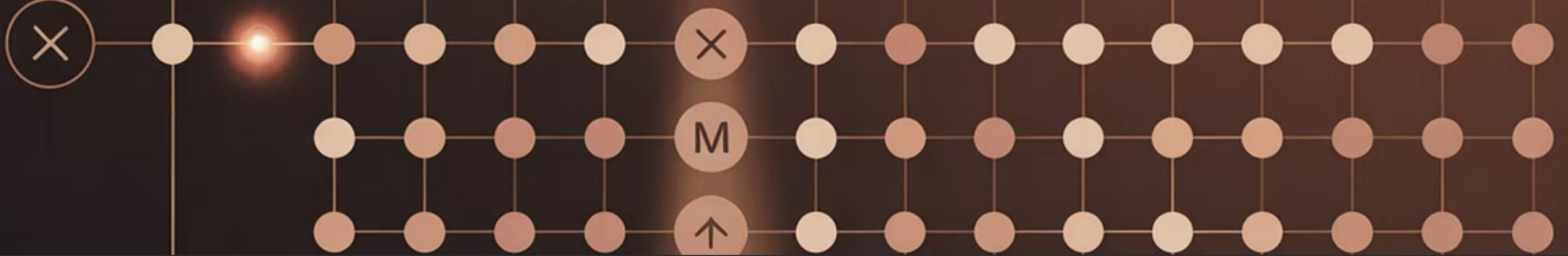
Árvore Geradora Mínima

Subgrafo que conecta todos os vértices do grafo original com custo total mínimo, sem formar ciclos.



Aplicações Práticas

Design de redes com custo mínimo, como redes elétricas, de telecomunicações ou de encanamento.



Algoritmo de Prim



Início

Começa com um único vértice e vai expandindo a árvore.



Expansão

A cada passo, adiciona a aresta de menor peso que conecta a árvore a um novo vértice.



Comparação com Kruskal

Prim cresce como uma única árvore. Kruskal permite várias árvores que eventualmente se fundem.

4

Melhor Cenário

Mais eficiente para grafos densos. Complexidade: $O(V^2)$ com matriz ou $O(E \log V)$ com heap.

Detectando ciclos em grafos

Em Grafos Não-Dirigidos

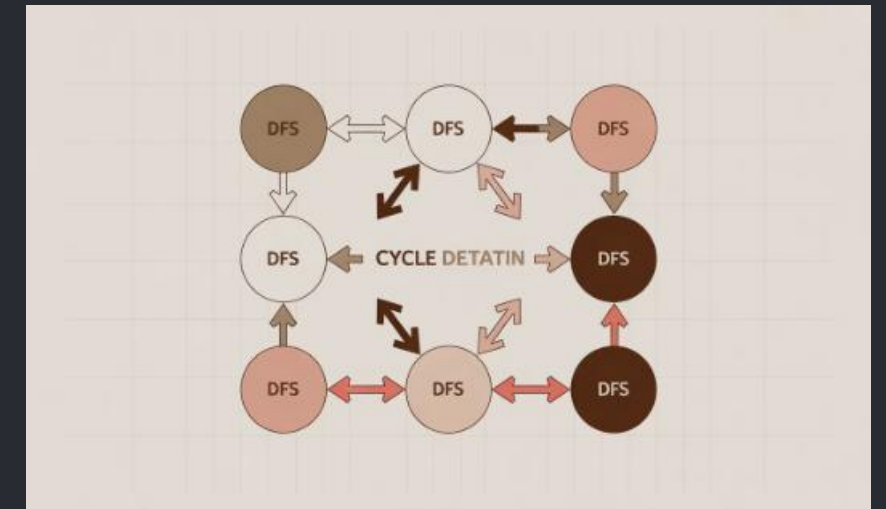
Um grafo não-dirigido tem ciclo se, durante uma DFS, encontramos uma aresta para um vértice já visitado (exceto o predecessor).

Aplicações: verificar se um grafo é uma árvore (conexo e sem ciclos).

Em Grafos Dirigidos

Usamos cores para marcar vértices: branco (não visitado), cinza (em processamento), preto (finalizado).

Um ciclo existe se encontrarmos uma aresta para um vértice cinza durante a DFS.





Componentes fortemente conexos



Definição

Um componente fortemente conexo (CFC) é um subgrafo máximo onde existe um caminho entre quaisquer dois vértices.

2

Algoritmo de Kosaraju

Utiliza duas DFS: uma no grafo original e outra no grafo transposto (direções invertidas).

3

Complexidade

$O(V+E)$, eficiente para análise de grandes redes dirigidas.

4

Aplicações

Análise de dependências em software, identificação de comunidades em redes sociais dirigidas.

Ciclos Eulerianos e Hamiltonianos

Ciclo Euleriano

Um caminho que percorre cada aresta exatamente uma vez, retornando ao vértice inicial.

Problema original das pontes de Königsberg (Euler, 1736).

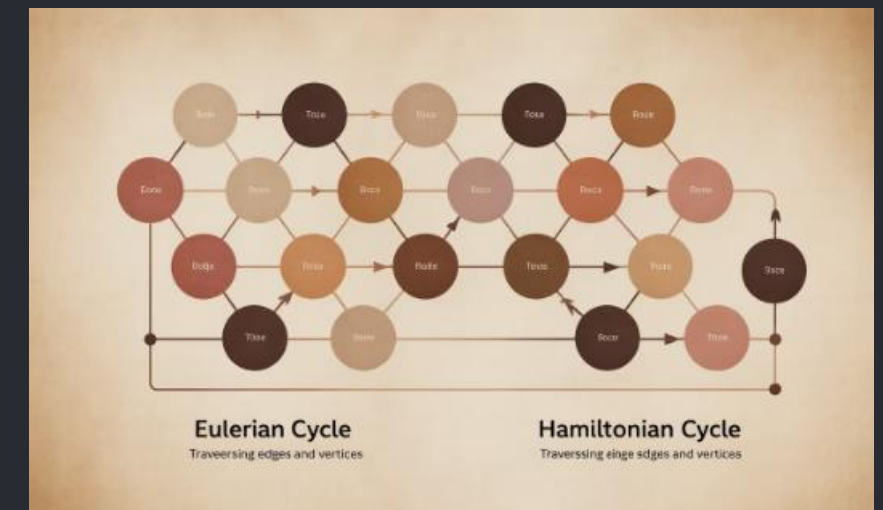
Condição: todos os vértices têm grau par.

Ciclo Hamiltoniano

Um caminho que visita cada vértice exatamente uma vez, retornando ao inicial.

Problema do caixeiro viajante é uma variante com pesos.

Não há condição simples para existência. É um problema NP-completo.



Exemplo: ciclo euleriano



Verificar Condições

Todos os vértices devem ter grau par e o grafo deve ser conexo

2

Algoritmo de Hierholzer

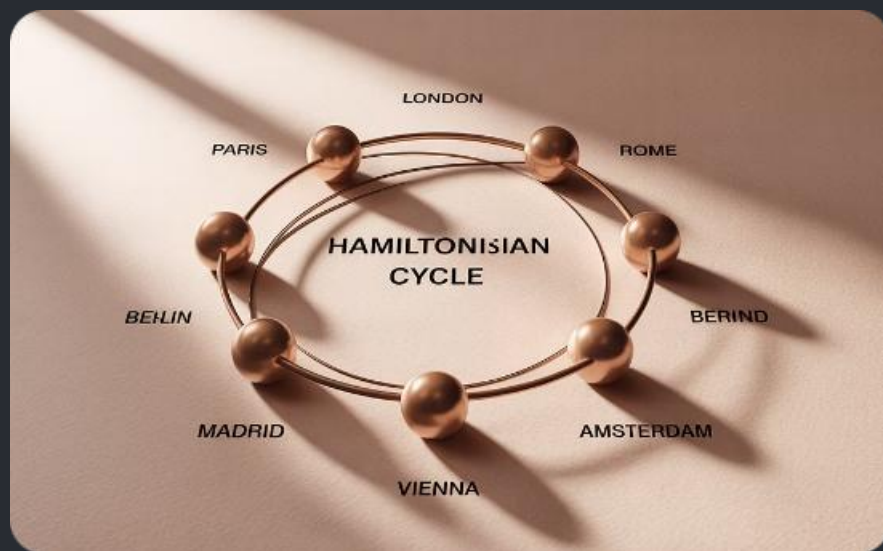
Encontra ciclos simples e os combina progressivamente

3

Exemplo Prático

O jogo de desenhar figuras sem levantar o lápis é um problema euleriano

Exemplo: ciclo hamiltoniano



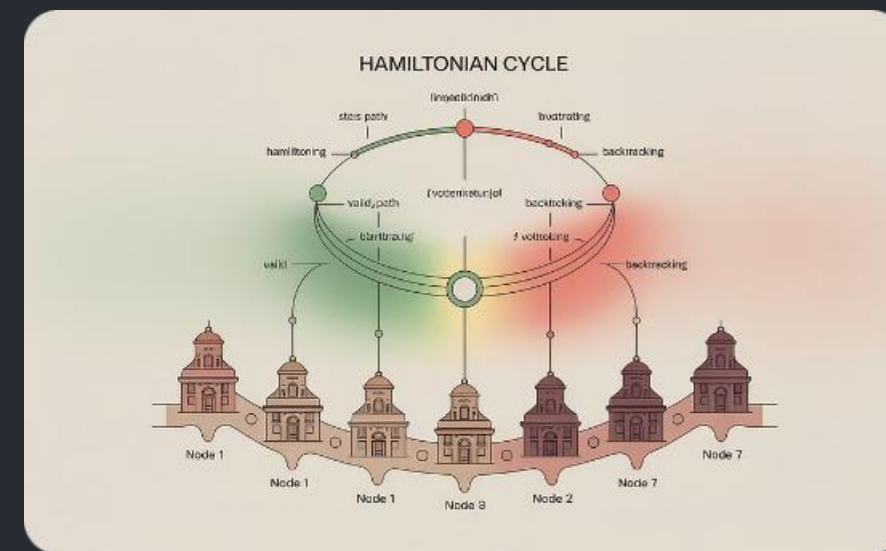
Problema do Caixeiro Viajante

Um ciclo hamiltoniano onde se busca minimizar a soma dos pesos das arestas. Aplicação clássica em logística e planejamento de rotas.



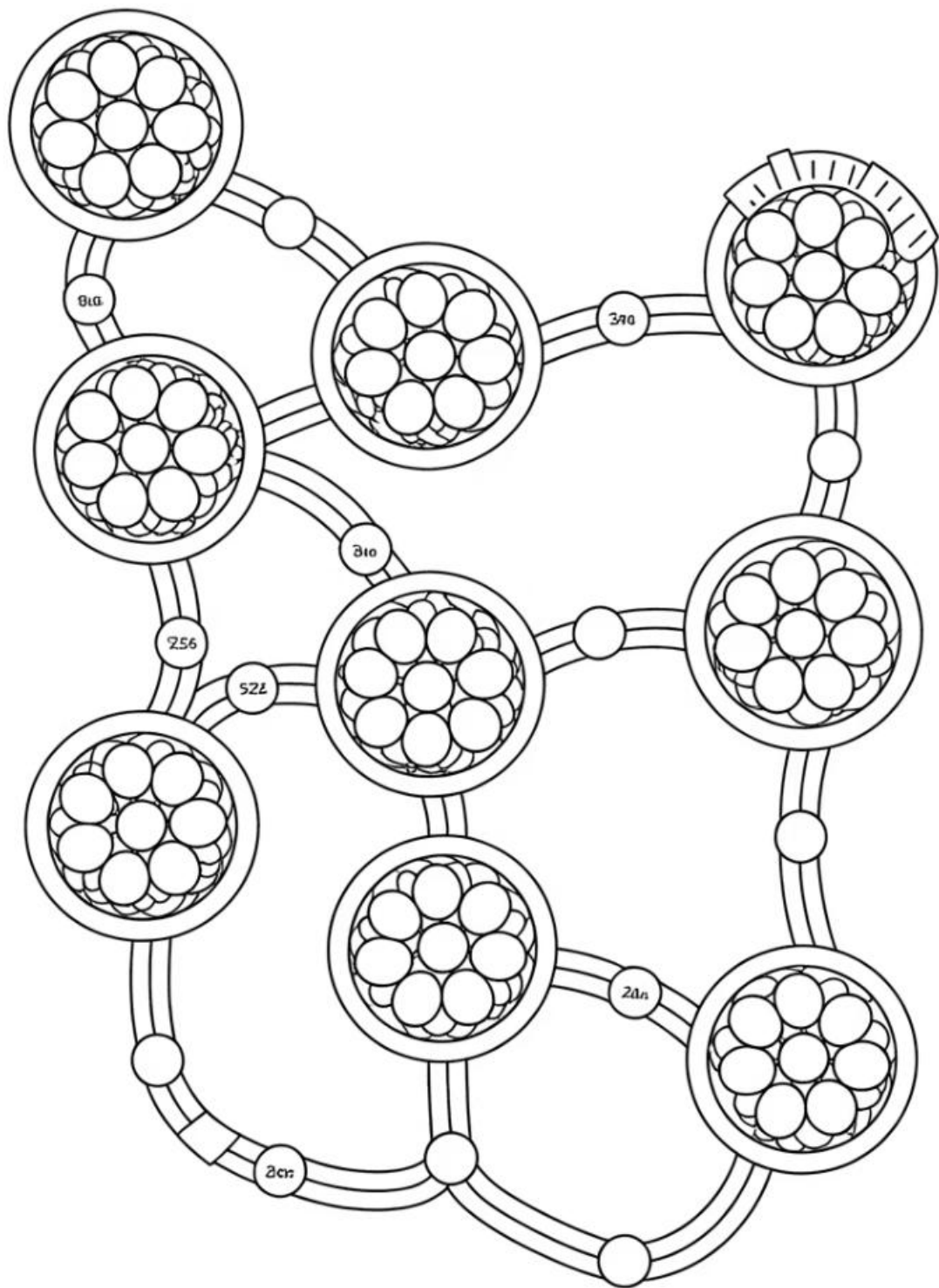
Passeio do Cavalo

Um caminho hamiltoniano em um grafo onde os vértices são as casas do tabuleiro de xadrez e as arestas são os movimentos válidos do cavalo.



Detecção Algorítmica

Geralmente requer força bruta ou backtracking. Não há algoritmo eficiente conhecido (problema NP-completo).



Graph Coloring Solution

Colorir grafos



Definição do Problema

Atribuir cores aos vértices de modo que vértices adjacentes tenham cores diferentes.

Número Cromático

O menor número de cores necessárias para colorir o grafo. Denotado por $\chi(G)$.

Aplicações

Escalonamento de horários, alocação de frequências, registros em compiladores.

Complexidade

Determinar o número cromático exato é NP-difícil. Existem heurísticas eficientes.

Algoritmos de coloração de grafos



Algoritmo Guloso

Percorre os vértices em alguma ordem e atribui a menor cor disponível.



DSATUR

Prioriza vértices com maior número de cores distintas na vizinhança.



Backtracking

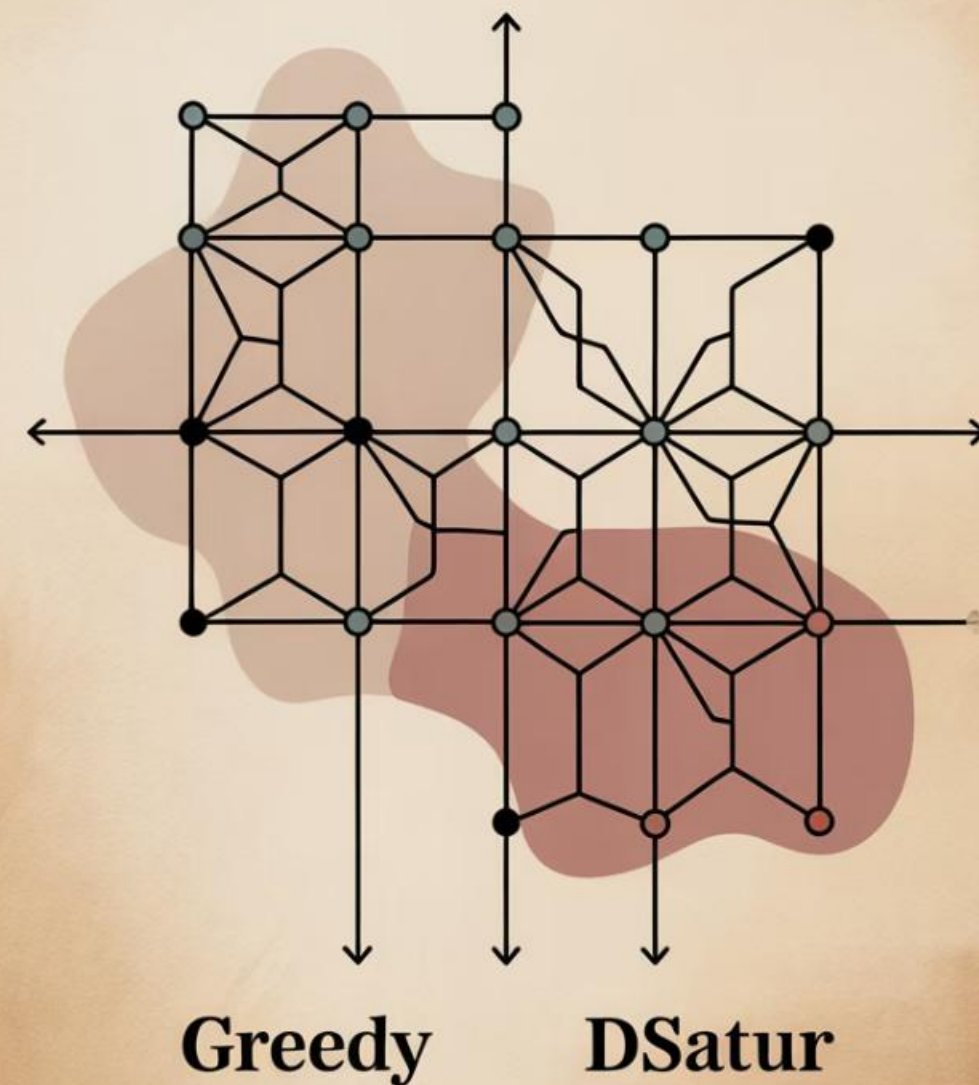
Abordagem exata que testa todas as possibilidades de coloração.



Algoritmos Genéticos

Abordagem evolutiva para encontrar colorações próximas do ótimo em grafos grandes.

Graph Coloring Algorithms



Grafos planares e teorema das quatro cores

Grafos Planares

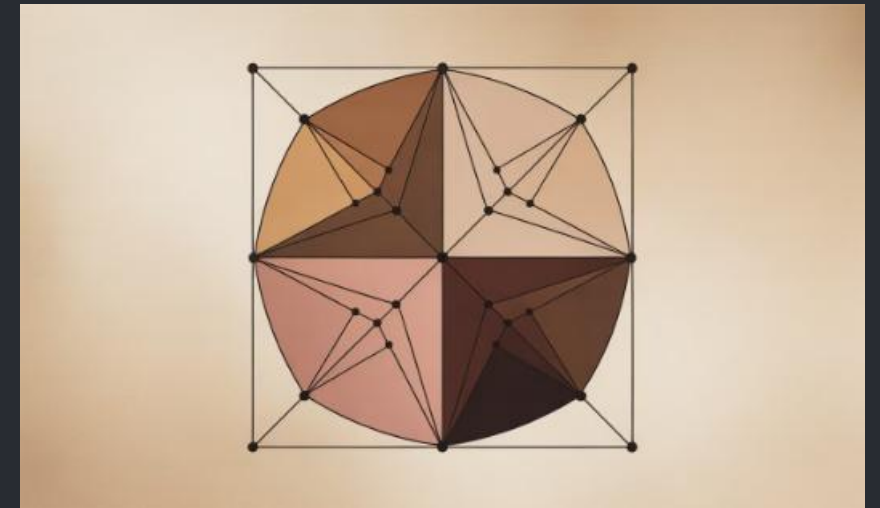
Um grafo é planar se pode ser desenhado em um plano sem que suas arestas se cruzem.

Exemplos: árvores, grafos outerplanar, grafos das faces de poliedros.

Teorema das Quatro Cores

Qualquer mapa planar pode ser colorido usando apenas quatro cores, de modo que regiões adjacentes tenham cores diferentes.

Primeiro teorema matemático provado com auxílio de computador (1976).



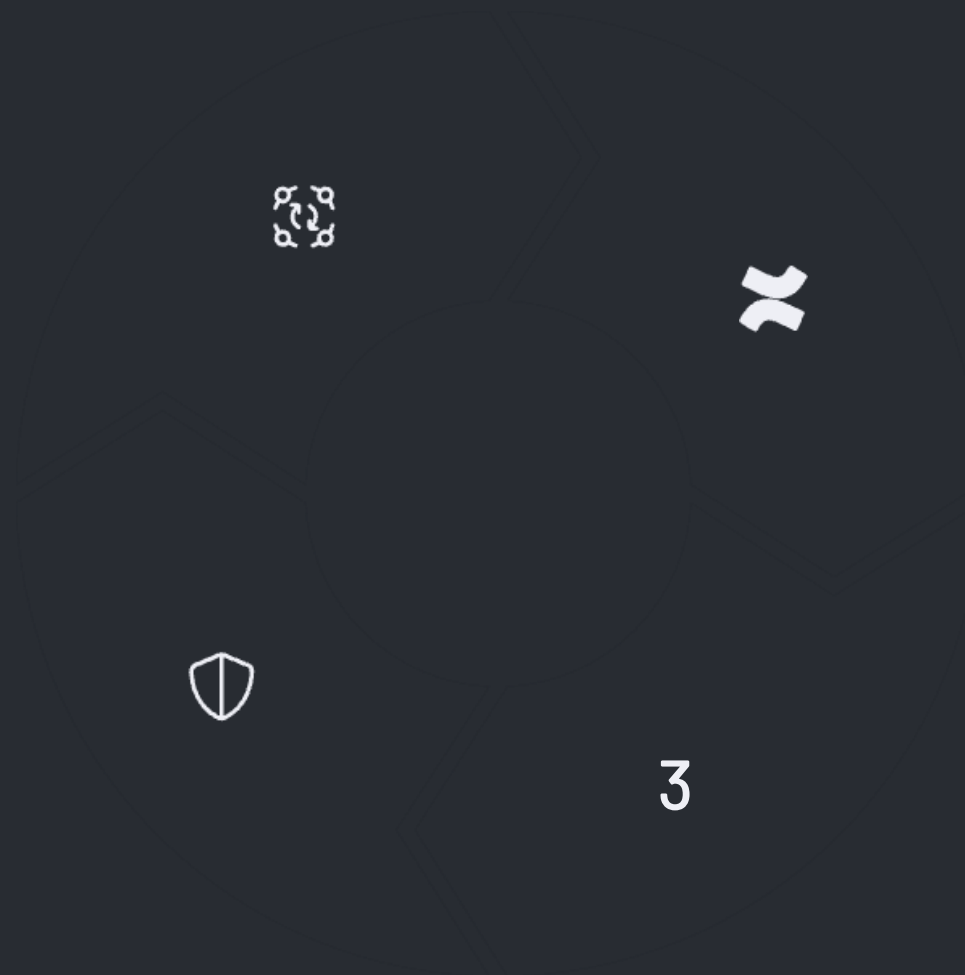
Casos de estudo: redes sociais

Detecção de Comunidades

Identificação de grupos fortemente conectados que podem representar comunidades de interesse.

Privacidade e Segurança

Análise de vulnerabilidades e proteção de informações em redes sociais.



Análise de Influência

Identificação de nós com maior centralidade ou capacidade de disseminação de informações.

Sistemas de Recomendação

Sugestão de amigos ou conteúdos baseados em conexões e comportamentos similares.

Casos de estudo: internet e web

Web como Grafo Dirigido

Páginas da web são vértices. Links são arestas dirigidas. Este modelo permite analisar a estrutura da web.

Algoritmo PageRank

Desenvolvido pelo Google, avalia a importância de páginas baseado na estrutura de links. Páginas importantes recebem links de outras páginas importantes.

Roteamento na Internet

Algoritmos de caminho mínimo determinam as rotas mais eficientes para pacotes de dados entre roteadores na rede.

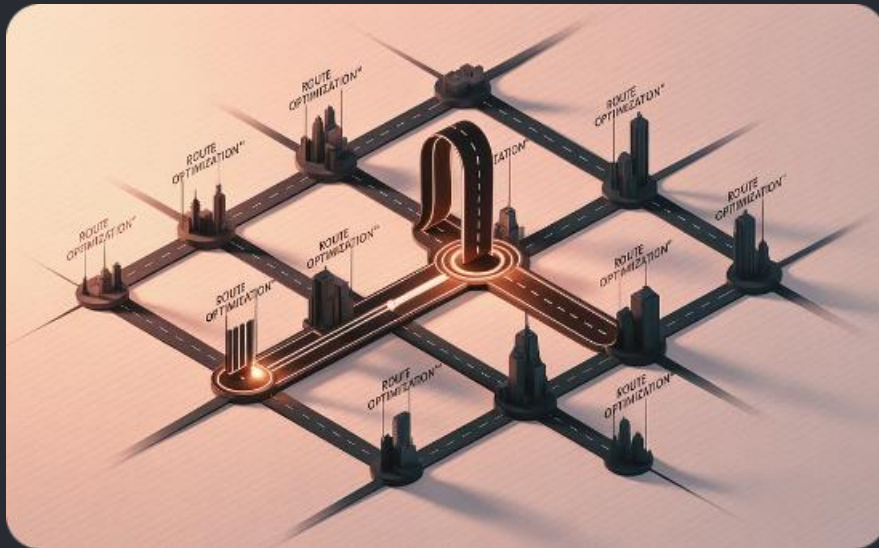
Detecção de Fraudes

Análise de padrões anômalos na estrutura de links para identificar sites maliciosos ou farms de links.



Pagerank

Casos de estudo: logísticas e transportes



Roteamento de Veículos

Aplicação de algoritmos de caminho mínimo para encontrar rotas ótimas em sistemas de navegação GPS, considerando distância, tempo e tráfego.



Planejamento de Transporte Público

Modelagem de linhas de ônibus, metrô e trem como grafos para otimizar conexões e minimizar tempos de transferência.



Logística de Distribuição

Otimização de rotas para entregas múltiplas, localização de centros de distribuição e análise da cadeia de suprimentos.



Protein

Casos de estudo: biologia computacional

Sequenciamento de DNA

Grafos de De Bruijn para montagem de fragmentos de sequenciamento genético.

Redes de Interação Proteína-Proteína

Modelagem de interações entre proteínas para entender funções biológicas e doenças.

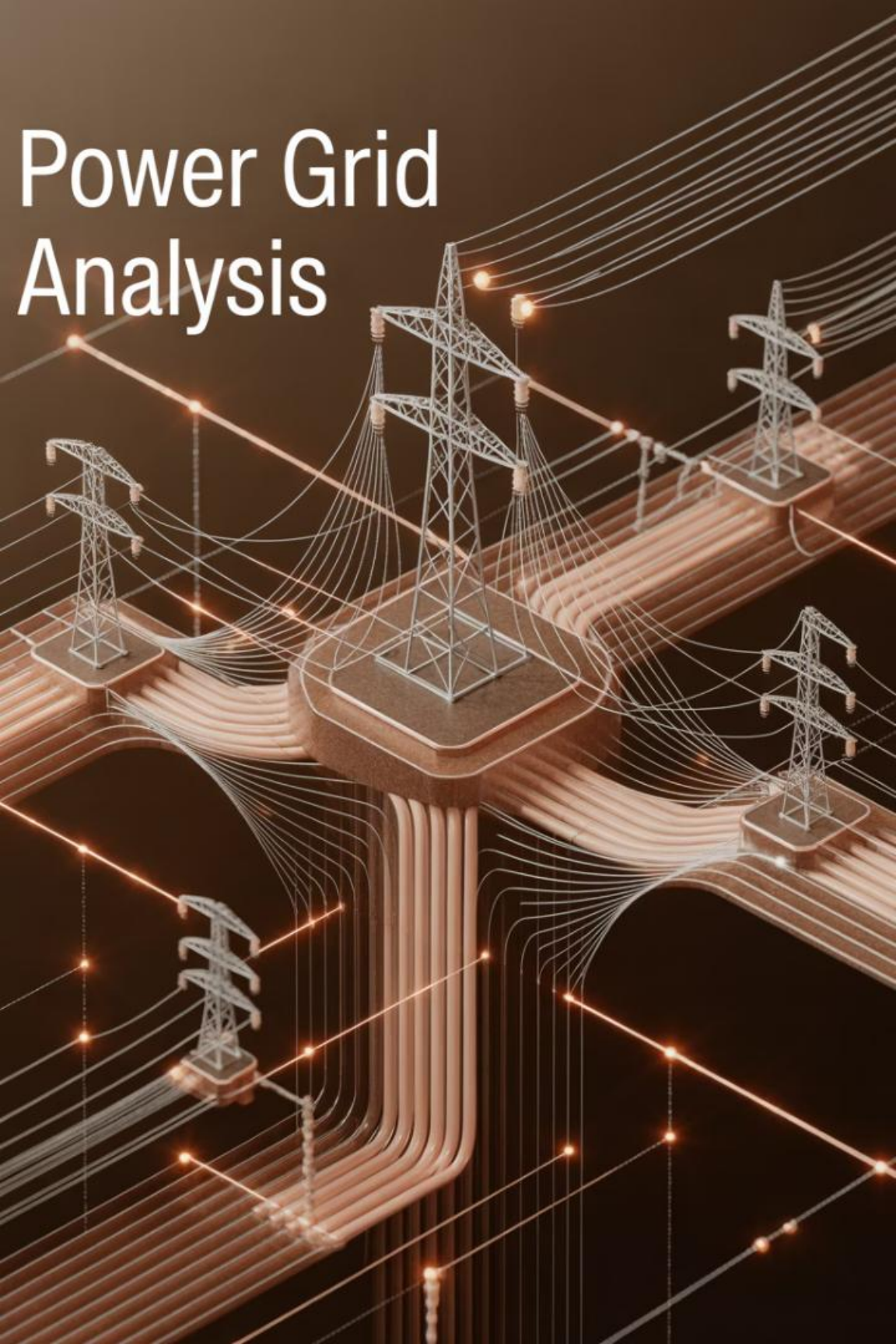
Vias Metabólicas

Representação de reações químicas em células como redes de transformação.



Filogenética

Árvores evolutivas modeladas como grafos para estudar relações entre espécies.



Power Grid Analysis

Casos de estudo: engenharia elétrica

99.9%

Confiabilidade de Rede

Redes elétricas redundantes garantem alta disponibilidade com caminhos alternativos.

30%

Redução de Perdas

Otimização de fluxo em redes de energia pode diminuir perdas de transmissão.

5x

Aumento de Eficiência

Análise de grafos em microchips melhora design de circuitos integrados.

Análise de redes complexas

Propriedades de Redes Reais

Redes complexas apresentam características como distribuição de grau seguindo lei de potência, alto coeficiente de clustering e baixo diâmetro.

Exemplos: internet, redes sociais, redes biológicas, redes de citações.

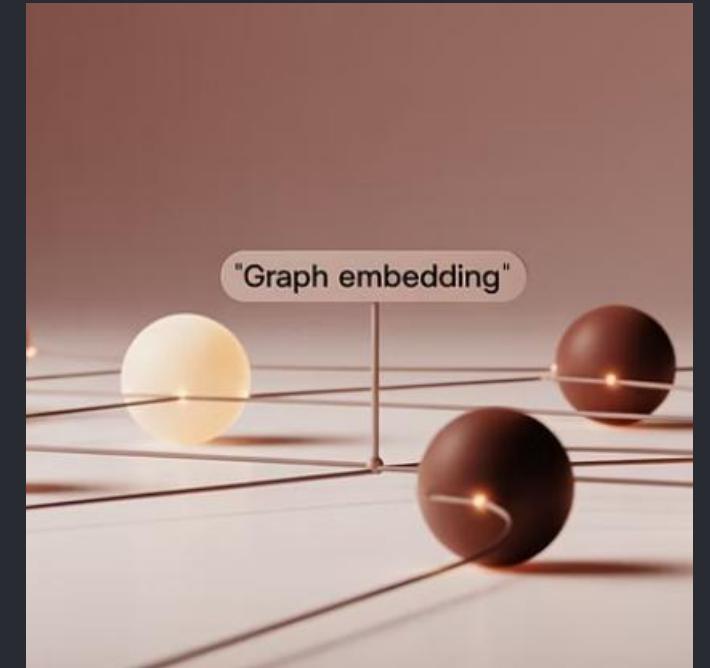
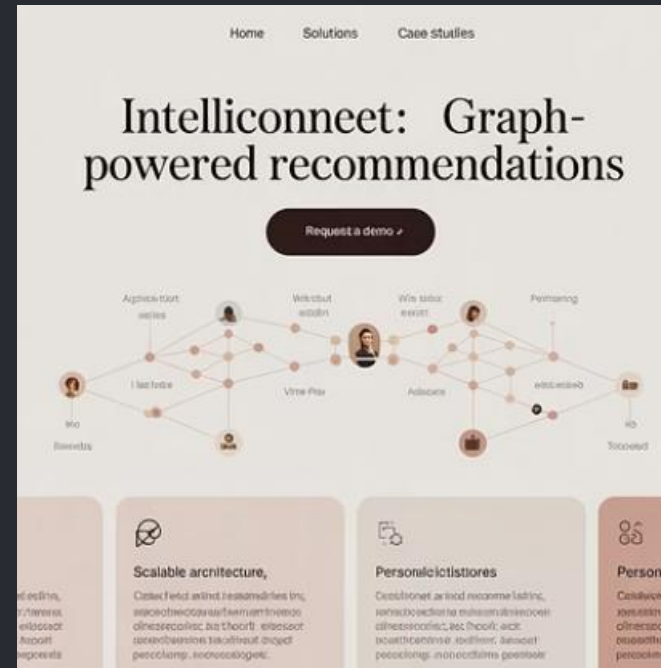
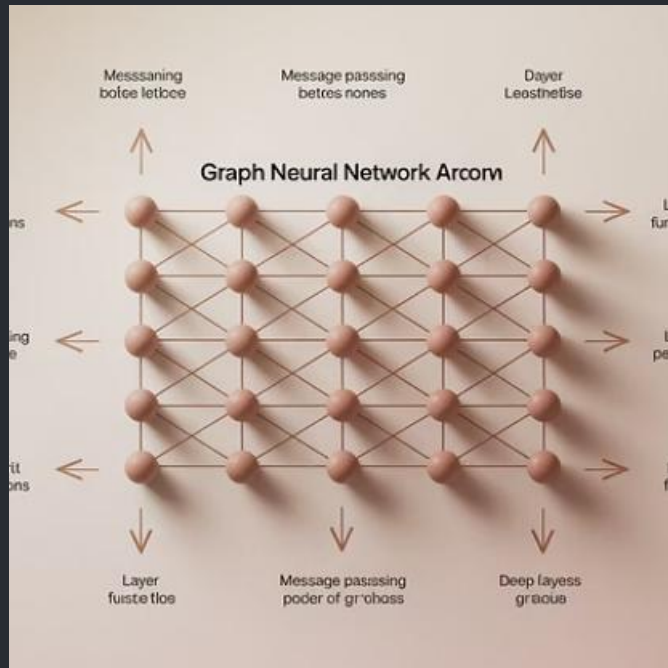
Fenômeno do Mundo Pequeno

A maioria dos nós em redes reais pode ser alcançada a partir de qualquer outro nó em poucos passos.

O famoso "seis graus de separação" é um exemplo deste fenômeno em redes sociais.



Grafos em IA e Machine Learning



Redes Neurais em Grafos (GNN)

Arquiteturas que operam diretamente em estruturas de grafos, aplicando convolução e propagação de mensagens.



Grafos de Conhecimento

Representações estruturadas de fatos que permitem raciocínio e inferência em sistemas de IA.



Sistemas de Recomendação

Modelagem de interações usuário-item como grafos bipartidos para recomendações personalizadas.

Algoritmos paralelos para grafos



Particionamento

Divisão do grafo em subgrafos que podem ser processados independentemente.



Distribuição

Alocação de partições em diferentes nós de processamento.



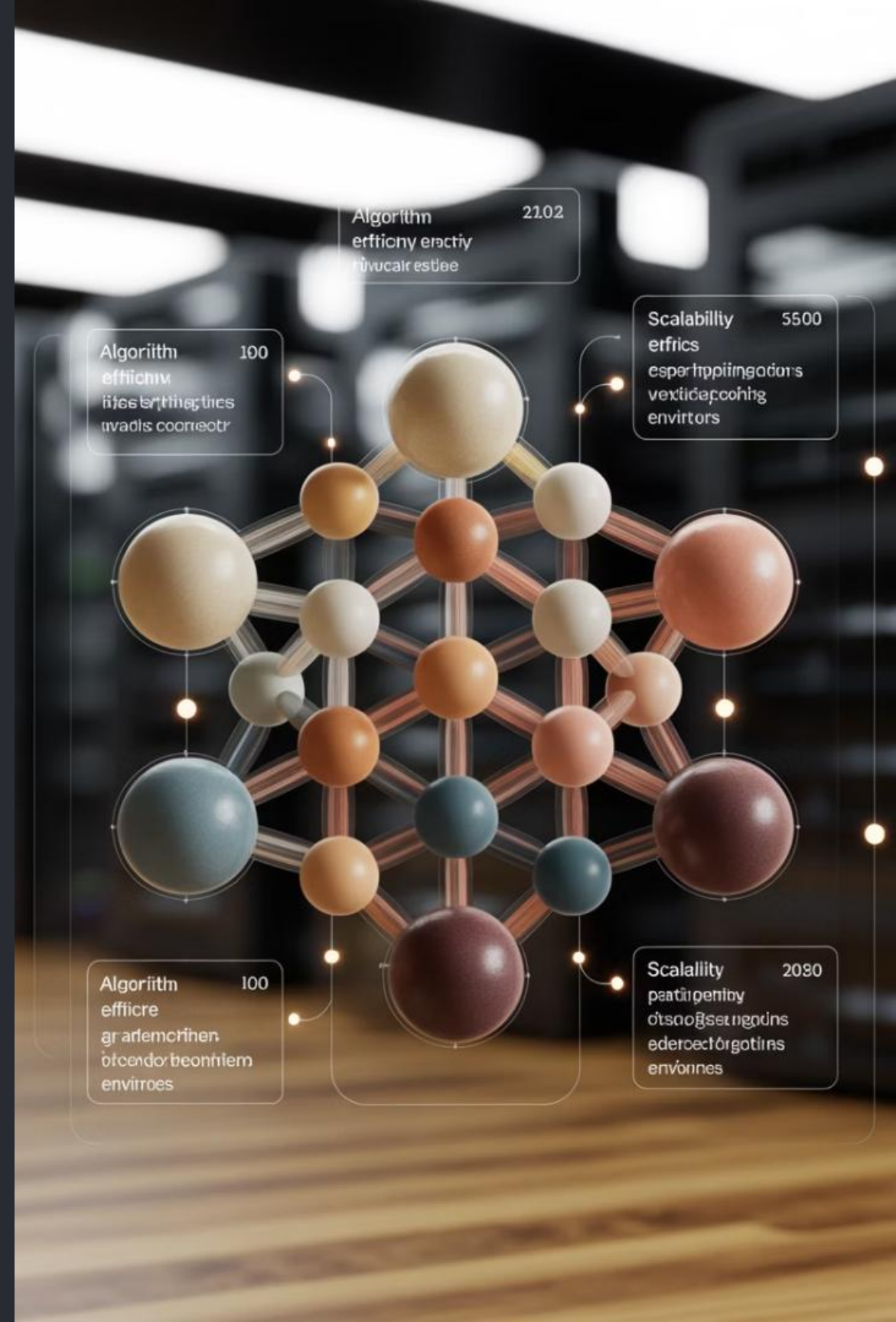
Computação

Processamento paralelo com comunicação entre bordas de partições.

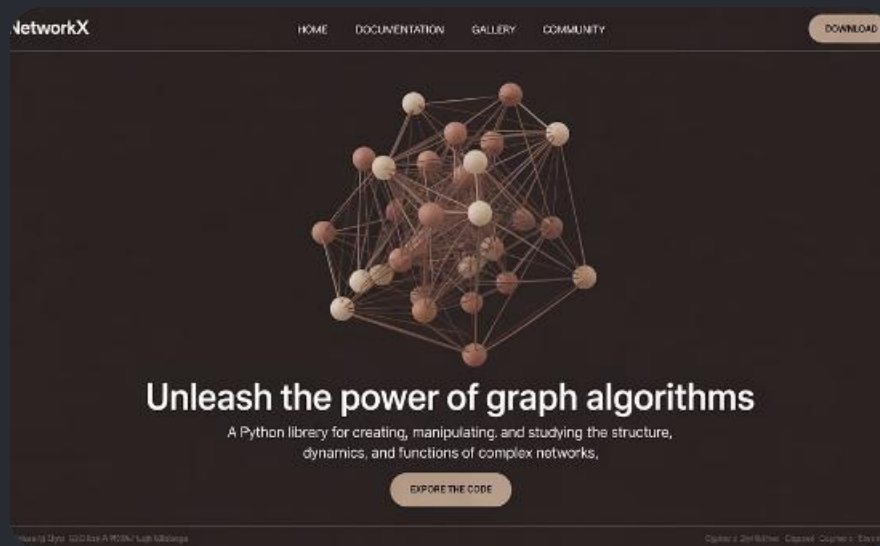


Combinação

Integração dos resultados parciais para formar a solução completa.



Software e bibliotecas para trabalhar com grafos



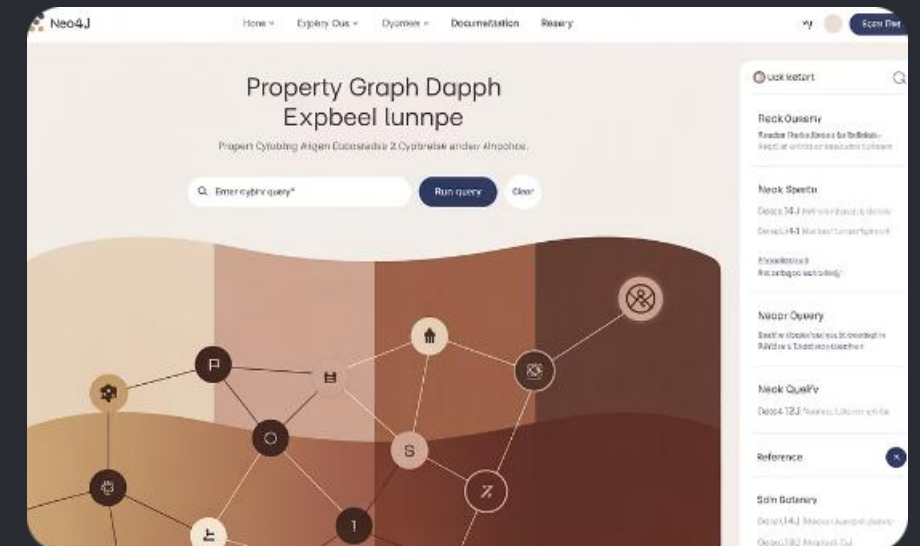
NetworkX(Python)

Biblioteca Python para criação, manipulação e estudo de estruturas, dinâmicas e funções de redes complexas.



Gephi

Plataforma interativa de visualização e exploração para grafos e redes. Ideal para análise visual e exploratória.



Neo4j

Banco de dados orientado a grafos com linguagem de consulta Cypher. Otimizado para armazenar e consultar grafos.

Considerações de desempenho em grafos massivos

1

Algoritmos Escaláveis

Complexidade sublinear ou aproximações eficientes

|0|

Processamento Distribuído

Frameworks como Spark GraphX e Pregel



Otimizações de Memória

Estruturas compactas e processamento em streaming



Aceleração por Hardware

GPUs e hardware especializado para grafos



Amostragem e Aproximação

Técnicas estatísticas para trabalhar com subconjuntos

Desafios atuais em teoria dos grafos



Detecção de Comunidades

Identificação precisa de grupos coesos em redes complexas, com aplicações em redes sociais e biologia.

Grafos Dinâmicos

Algoritmos para grafos que mudam ao longo do tempo, mantendo análises eficientes durante as mudanças.

Escalabilidade em Big Data

Processamento de grafos com bilhões de vértices e arestas, como web e redes sociais completas.

Incorporação de Grafos

Representações vetoriais eficientes de grafos para machine learning e análise avançada.

Ética e responsabilidade nas aplicações de grafos

Privacidade em Redes Sociais

- Proteção de dados pessoais em análises de rede
- Anonimização de grafos preservando propriedades estruturais
- Consentimento informado para coleta e análise de dados relacionais

Viés em Algoritmos

- Equidade em sistemas de recomendação baseados em grafos
- Transparência em algoritmos de ranqueamento como PageRank
- Detecção e mitigação de viés em análise de redes

